基于历史误差场景与滚动优化的微网购电及储能调控策略

摘　要

微网调控需要协调光伏出力、用电负荷和储能状态，在满足供电需求的同时控制计划交易、紧急补电及日内调整费用。针对预测误差与电价变化，本文构建由单日确定性调度、历史误差场景计划和日内滚动调整组成的优化框架，并利用 2025 年数据进行连续运行检验。

针对问题 1，以购电费用最小为目标，将功率平衡、充放电效率、容量与功率限制及日初日末电量相等条件写为线性约束，采用 HiGHS 求解器求解。允许合理处理盈余电量，并通过保持储能状态不变的等价变换消除同时充放电。所得全天购电量为 59482.70 千瓦时，购电费为 35126.95 元，较无储能就地消纳方案降低 26.90%。

针对问题 2，先利用 1 月份数据选择历史预测方法，再将近期净负荷误差构造成场景，把五倍电价的缺额惩罚纳入日前目标。运行时根据实际净功率进行充放电安全裁剪与紧急补电，并将日末实际电量传递至次日。2 月至 12 月总费用为 13863865.66 元，紧急购电量为 85480.79 千瓦时，较点预测日前基准降低总费用 13.34%。

针对问题 3，在四个预报发布时刻更新光伏信息，针对未执行时段重新优化，以当天零时计划统一计算偏差费用。在“原计划电费不退还、未来电价需预测”的主口径下，总费用为 13934495.57 元，较仅使用零时预报降低 0.54%，紧急购电量降低 46.80%；但相较问题 2 总费用略增，表明更新收益需与交易成本共同衡量。

针对问题 4，分别检验电价需要预测和日前电价已知两种信息设定。可实施的价格预测方案中，日前与滚动策略总费用分别为 14621415.17 元和 14711757.51 元。进一步比较退购结算解释与场景窗口，揭示费用口径对策略排序的影响。全部主模型均满足逐时功率平衡、储能边界与跨日连续性；新增反馈对照显示，简单保留规则在本样本中反而增加费用，故不把它作为主策略。

关键词：线性规划；历史误差场景；滚动优化；储能调度；信息可得性

一、 问题背景与重述

1.1 问题背景

1.1.1 微网结构与调控目标

微网通过协调小区负荷、光伏发电、储能设备和外部电网，实现分布式能源的本地利用。光伏出力具有明显的日内变化，用电负荷与电价的峰谷也不完全同步，因此仅凭当前供需差额购电，难以充分发挥储能的经济价值。合理的调控策略应在低价或光伏富余时补充储能，在负荷较高或购电较贵时释放电能，同时保留对预测误差的应对能力。

1.1.2 调度尺度与交易机制

本文以 10 分钟为一个调度时段，每日划分 144 个时段。日前计划确定全天常规购电安排；当实际供电不足时，按交易时刻电价的 5 倍紧急补电；获得新光伏预报后，可针对未执行时段调整计划，并承担相应偏差费用。购电量与储能电量均以 kWh 计，功率以 kW 计，费用以元计。

1.2 四个问题的核心任务

1.2.1 问题 1：单日确定性调度

依据附件 1 给出的日内电价、负载和光伏预测，制定一天的购电及储能充放电计划。在供电满足负荷、储能安全运行且日初日末电量均为 6000 kWh 的条件下，最小化购电费，并给出题目指定时段的电量、全天费用以及完整结果文件。

1.2.2 问题 2：全年日前计划与紧急补电

在每天重复使用附件 1 日内电价曲线的条件下，处理全年负荷和光伏实际出力变化。每天零时只能利用当时已有的信息制定计划，运行中对供电缺额紧急补电。需要连续模拟储能状态，报告四个指定日期和 2025 年 2 月 1 日至 12 月 31 日的完整结果。

1.2.3 问题 3：预测更新与购电调整

利用每天 0、6、12、18 时发布的未来 24 小时整点光伏预报，更新尚未执行时段的购电安排。在统一计划基准下处理上调购电和下调违约费用，比较不同预报使用频率的成本，判断引入日内更新的实际价值。

1.2.4 问题 4：波动电价下的策略比较

采用附件 4 的逐日电价重新计算问题 2 和问题 3。除替换结算价格外，还需明确未来电价能否在日前获知。本文以历史价格预测驱动的策略作为可实施主方案，并设置当天电价已知的对照，分析价格信息对购电、储能与总费用的作用。

二、 问题分析

2.1 整体思路及四问联系

2.1.1 由基础物理模型到动态决策

四个问题共用功率平衡和储能动态约束。问题 1 提供无预测偏差条件下的基础模型；问题 2 增加全年运行跨度、历史预测和紧急补电惩罚；问题 3 进一步增加光伏预报更新及计划调整费用；问题 4 在同一物理框架上考虑逐日电价变化和价格信息限制。四问分别检验模型的经济性、连续性、适应性与信息依赖程度。

需要区分两类概念：问题 1 至 3 使用的电价并非全天常数，而是每天重复的**日内变化曲线**；问题 4 的电价则随日期和时段共同变化。另外，已有一条波动价格曲线不等于未来价格在决策时已知。将这两点分开，有助于避免把事后信息优势误认为优化算法的改进。

2.1.2 预测、计划、执行与评价闭环

基本流程如图1所示。预测层产生未来净负荷及误差场景；计划层确定常规购电和名义储能轨迹；执行层依据实际供需进行安全反馈；评价层使用实际交易价格核算费用并更新历史误差。真实未来数据仅用于事后评价，不能反向参与已生成的可实施计划。

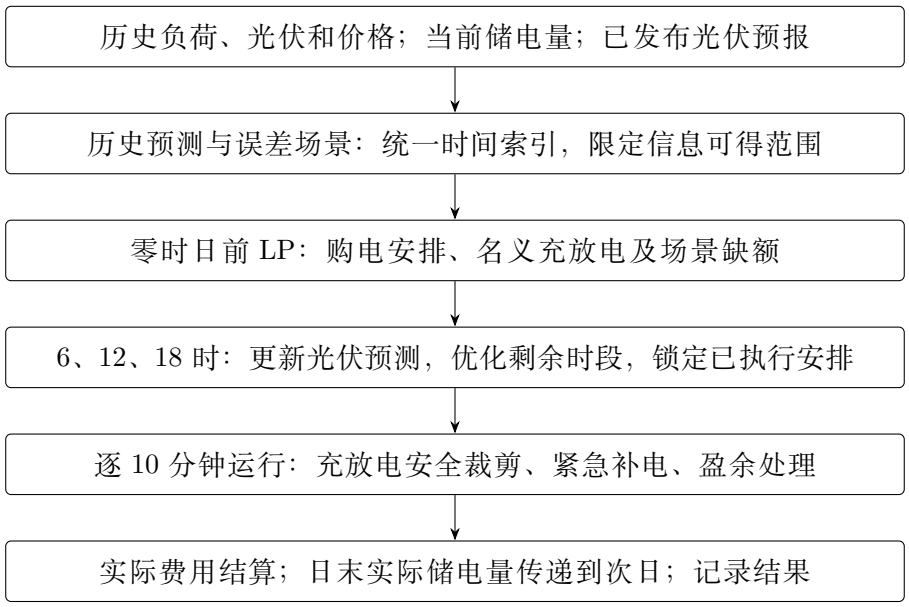


图 1 统一建模与运行流程；问题 1 不执行误差场景和日内更新步骤

2.2 各问题的关键难点

2.2.1 问题 1：功率、效率与储能边界

储能约束将不同时段耦合，充电输入和放电输出又存在效率损失，因此不能仅按价格排序独立决策。还应区分功率值与时段电量，并明确储电量属于时段边界状态。光伏富余时若既不允许售电又不允许弃电，单纯的等式平衡可能导致不必要的不可行或耗能循环，需要建立明确的盈余处理机制。

2.2.2 问题 2：日前未知量与跨日连续性

附件 2 提供实际量而非日前预测。直接把当天全部实际负荷和光伏输入零时优化，会利用未来信息，并削弱紧急购电问题本身的意义。本文从历史数据构造预测，把缺额风险纳入目标，再将**实际执行后的电量**作为次日初值。名义日末状态只用于当天优化，不能替代实际状态。

2.2.3 问题 3：预测发布时序与费用基准

同一未来时段可能经历多轮优化。若每次都把新旧计划差异重复计入全年费用，将产生重复结算。本文固定以零时计划为基准，只对最终生效的常规购电量结算一次偏差费用。退购是否退还原电费存在解释空间，因此主模型与敏感性模型分别采用两种明示口径。

2.2.4 问题 4：价格信息与结果可比性

波动电价影响原计划费、紧急费及调整费的全部系数。可实施策略只能使用历史预测价格，而实际费用必须用实际价格计算。比较方案时还需要报告统计期初末储能状态，防止把消耗额外初始库存形成的成本差异全部归因于策略优越性。

三、 模型假设

3.1 物理运行假设

3.1.1 离散功率与能量损耗

假设每个 10 分钟时段内功率保持恒定，$\Delta t = 1/6$ h。题面只给出 90% 效率而未明确是单程还是回路效率，本文将其解释为充电效率与放电效率分别为 $\eta_c = \eta_d = 0.9$，故完整充放电回路效率为 0.81；除储能效率损失外，忽略线路损耗、自放电、启停损耗及辅助设备耗电。充放电量采用交流母线侧口径。

3.1.2 单向购电与盈余处理

外网连接只允许非负购电，不引入未给出的售电收益及购电功率上限。光伏可优先分配给负荷，剩余用于充电；超过负荷和可充电能力的供给允许弃光或不予利用。已经承诺的购电电量仍按相应合同规则付费。盈余变量不代表可售电量，也不产生收益。实际执行阶段不同时充放电。

3.2 决策与交易假设

3.2.1 信息可得性与预测使用

每次决策可获得当前储电量、此前实际数据和已经发布的预报。未发生的负载、实际光伏与实际价格不能用于可实施计划。附件 3 整点预报以线性插值扩展到 10 分钟尺度，插值起点使用发布时刻刚完成的时段观测。1 月份用于初始化和预测方法选择，2 月至 12 月为报告期，不以报告期结果重新选择主预测方法。

3.2.2 结算基准与边界条件

问题 1 固定日初日末均为 6000 kWh；问题 2 至 4 只固定 2025 年 1 月 1 日零时初值为 6000 kWh，不附加逐日循环边界。主结算口径保留原计划全额电费，并增加上调费用及下调违约费；另一口径退回取消电量的原电费后收取 50% 违约费。两种口径均以当天零时计划为基准，实际紧急购电另按 5 倍交易电价结算。对购电不足的实时反馈优先维持负荷供给，不为追踪名义充电计划额外紧急购电。

四、 符号说明

4.1 索引、输入参数与决策变量

4.1.1 日期、时段及输入参数

令日期索引 $d = 1, \ldots, 365$，报告期为 $\mathcal{D} = \{32, \ldots, 365\}$；时段索引 $t = 1, \ldots, 144$，$t = 1$ 对应 0:00–0:10，状态 $E_{d,t}$ 为第 t 个时段结束时的电量，$E_{d,0}$ 为零时电量。发布时刻 $k \in \{0, 6, 12, 18\}$，误差场景索引为 s。单日推导省略日期下标。

符号	说明	单位
$P_{\mathrm{load},d,t}, P_{\mathrm{pv},d,t}$	实际负载功率、实际光伏功率	kW
$\widehat{P}_{\mathrm{load},d,t}$	当日零时形成的负载预测	kW
$\widehat{P}^{(k)}_{\mathrm{pv},d,t}$	发布时刻 k 对应的光伏预测	kW
$n_{d,t}, \widehat{n}^{(k)}_{d,t}$	实际净负荷、预测净负荷，均为负载减光伏	kW
$n^{(k)}_{d,t,s}$	场景净负荷	kW
$\lambda_{d,t}, \widehat{\lambda}_{d,t}$	实际交易电价、日前预测电价	元/kWh
η_c, η_d	充电、放电效率，均为 0.9	–
Δt	时间步长，等于 1/6	h
$E_{\min}, E_{\max}$	允许储电量下限 1200、上限 10800	kWh
$P_{\max}$	最大充电、放电功率，均为 5000	kW
$P^{0}_{\mathrm{grid},d,t}$	零时原始计划购电功率	kW
$P_{\mathrm{grid},d,t}$	最终生效的常规购电功率，不含紧急补电	kW
$P_{\mathrm{ch},d,t}, P_{\mathrm{dis},d,t}$	母线侧充电输入、放电输出功率	kW
$E_{d,t}$	实际或模型中明确指定的边界储电量	kWh
$P_{\mathrm{em},d,t}$	实际紧急购电功率	kW
$P_{\mathrm{sp},d,t}$	弃光或未利用的已付费供给功率	kW
$u_{d,t,s}$	计划模型中的场景缺额补救变量	kW
$y^{+}_{d,t}, y^{-}_{d,t}$	相对零时计划的上调、下调功率	kW
C_{total}	报告期实际总费用	元

4.1.2 计划状态与实际状态的区分

优化阶段的充放电变量与状态描述名义储能计划；实际执行时，以反馈规则重新计算充放电与状态。必要时分别加上标 pl 和 act。场景缺额 u 只是优化目标中的风险代理量，不与实际紧急功率 P_{em} 混用。

4.2 费用与储能利用指标

4.2.1 费用分项

定义原计划费用 C_0、上调费用 C_+、下调净费用 C_- 与实际紧急费用 C_{em}，总费用为

$$C_{\mathrm{total}} = C_0 + C_+ + C_- + C_{\mathrm{em}}. \tag{1}$$

主口径中 $C_- \geq 0$；退费口径下 $C_- \leq 0$，表示退费与违约费合并后的净减少额，不应解释为负的违约单价。

4.2.2 等效循环指标

为统一表征储能使用强度，定义基于 9600 kWh 可用容量的能量吞吐等效循环次数

$$N_{\mathrm{eq}} = \frac{\sum_{d\in\mathcal{D},t}\left(\eta_c P_{\mathrm{ch},d,t} + P_{\mathrm{dis},d,t}/\eta_d\right)\Delta t}{2(E_{\max} - E_{\min})}. \tag{2}$$

该量是运行强度指标，不直接等同于寿命模型中的电池循环计数，也不用于假设额外的折旧成本。

五、 数据处理与分析

5.1 数据统一与时间对齐

5.1.1 附件结构与完整性检验

附件 1 有 144 行 10 分钟数据；附件 2 的负荷与实际光伏各有 365 天、每天 144 个数值；附件 3 共 1460 条预报，每条包含 24 个未来整点值；附件 4 有 365 组 144 时段电价。程序核查数组维度、日期唯一性、数值有限性和非负性，所读数值均通过检查。附件 1 电价范围为 0.3713–1.3952 元/kWh，附件 4 范围为 0.0076–1.7936 元/kWh。

5.1.2 时段映射与模板标签修正

原始输入的时间字段从 0:10 到次日 0:00，共 144 个值；本文按区间末端标记解释，即 0:10 对应 0:00–0:10 的平均功率。附件 5 的部分模板却从 0:10–0:20 开始，末行延伸到次日 0:10，与完整自然日边界存在 10 分钟错位。输出保留原有工作表名称及字段结构，将 144 个时段标签统一为 0:00–0:10 至 23:50–24:00，不平移数值。此处理属于明确的时间离散化约定；若另有权威时标解释，应据此重新映射并计算。

对功率 P_t，时段电量为 $q_t = P_t\Delta t$。例如 4 小时分组包含 24 个时段，充电量为对应 24 个充电功率之和乘以 1/6。不能把 10 分钟功率直接填入以 kWh 为单位的表格。

5.2 历史预测与预报处理

5.2.1 预测方法选择与报告期评价

采用时间顺序验证，而非随机打乱训练与验证数据 [5]。候选方法为前一日持久性、3 日和 7 日移动平均、14 日指数加权平均以及过去最多四个同星期日期的均值。指数权重为 $\exp(-j/5)$，其中 j 表示距最新一天的天数。以 1 月 15 日至 31 日的逐日历史预测 MAE 选取方法，2 月 1 日起固定方法形式并滚动更新历史输入。1 月份运行热启动统一采用持久性方法，1 月 1 日历史不足时使用附件 1 提供的曲线作为初始先验，避免事后选型改变 1 月份运行状态。

对任一待预测序列 $x_{d,t}$，选出的同星期预测与 7 日均值分别为

$$\widehat{x}^{\text{week}}_{d,t} = \frac{1}{|\mathcal{L}_d|}\sum_{\ell\in\mathcal{L}_d} x_{d-\ell,t}, \quad \mathcal{L}_d = \{7,14,21,28\}\cap[1,d-1], \tag{3}$$

$$\widehat{x}^{\text{mean7}}_{d,t} = \frac{1}{m_d}\sum_{\ell=1}^{m_d} x_{d-\ell,t}, \quad m_d = \min(7,d-1). \tag{4}$$

历史不足一周时，同星期方法退化为已有最近七日的平均。预测评价结果见表2。WAPE 使用绝对误差总和除以实际值总和，避免夜间光伏零值使 MAPE 无定义。

表 2 历史预测方法的验证与测试误差

变量	选定方法	1 月验证 MAE	2–12 月 MAE	2–12 月 RMSE	2–12 月 WAPE
负载/kW	同星期均值	117.84	188.88	262.06	4.09%
光伏/kW	7 日移动均值	96.18	153.40	303.86	6.45%
电价/(元/kWh)	同星期均值	0.03	0.0489	0.0696	6.45%

负载与价格表现出明显的星期模式，光伏则采用近 7 日均值更合适。预测误差虽较小，仍可能在电量紧张的连续时段积累，因此需进一步把误差风险与储能状态共同纳入计划。

5.2.2 光伏整点预报插值及误差场景

问题 3 在发布时刻 k 读取相应预报行，令发布时刻观测为第 0 个节点，其后 24 个整点预报为第 1 至 24 个节点，对剩余 10 分钟区间末端线性插值。不同发布时刻分别构造误差历史，不能把 18 时发布的数据用于 12 时决策。负载预测在零时生成，日内不额外使用未来实际负载。

令 $\varepsilon_{j,t}^{(k)} = n_{j,t} - \widehat{n}_{j,t}^{(k)}$ 为过去日期 j 相同发布时刻的净负荷预测误差。主模型采用过去 $H = 14$ 天误差轨迹，加上一个零误差场景，得到

$$n_{d,t,s}^{(k)} = \widehat{n}_{d,t}^{(k)} + \varepsilon_{j_s,t}^{(k)}, \qquad \varepsilon_{j_0,t}^{(k)} = 0. \tag{5}$$

场景保留一条历史日内误差轨迹的形状，采用等权重，成熟阶段场景数 $S = 15$。这是近期误差经验分布的近似，不意味着所有未来误差均被覆盖，也不额外假定正态分布。后文以 7 天和 28 天窗口检验其敏感性。

六、 模型的建立与求解

6.1 问题 1：单日确定性线性规划模型

6.1.1 决策变量与购电费用目标

单日模型的主要变量为 $P_{\mathrm{grid},t}$、$P_{\mathrm{ch},t}$、$P_{\mathrm{dis},t}$ 与 E_t，并引入非负盈余处理量 $P_{\mathrm{sp},t}$。购电只发生在外网方向，目标函数为

$$\min C_1 = \sum_{t=1}^{144} \lambda_t P_{\mathrm{grid},t} \Delta t. \tag{6}$$

价格与时长均是已知常数，因此目标对购电功率线性。模型不引入题目未给出的电池寿命费、售电收益或峰值需量费。

6.1.2 功率平衡与储能约束

母线功率平衡及储能动态分别为

$$P_{\mathrm{grid},t} + P_{\mathrm{pv},t} + P_{\mathrm{dis},t} = P_{\mathrm{load},t} + P_{\mathrm{ch},t} + P_{\mathrm{sp},t}, \tag{7}$$

$$E_t = E_{t-1} + \eta_c P_{\mathrm{ch},t} \Delta t - \frac{P_{\mathrm{dis},t} \Delta t}{\eta_d}. \tag{8}$$

式(7)使富余供给具有明确去向；消去 $P_{\mathrm{sp},t} \geq 0$ 后，可等价写为供给不低于负荷和充电需求的不等式。式(8)中的充电量需乘效率，放电输出需除以效率，且始终使用 E_{t-1} 与 E_t 连接相邻边界。

$$1200 \leq E_t \leq 10800, \qquad t = 0, \ldots, 144, \tag{9}$$

$$0 \leq P_{\mathrm{ch},t} \leq 5000, \qquad 0 \leq P_{\mathrm{dis},t} \leq 5000, \tag{10}$$

$$P_{\mathrm{grid},t} \geq 0, \qquad P_{\mathrm{sp},t} \geq 0, \tag{11}$$

$$E_0 = E_{144} = 6000. \tag{12}$$

上述目标、等式和不等式均为线性表达式，故在暂不添加乘积互斥约束的情况下，可构成线性规划 [1]。

6.1.3 同时充放电的处理与线性求解

不能仅凭电价为正就无条件断言任何 LP 最优解均不同时充放电。本文在允许零成本盈余处理且效率为常数的条件下，给出保持状态的显式消除方法。设某时段 $c = P_{\mathrm{ch},t} > 0$、$p = P_{\mathrm{dis},t} > 0$，取

$$\delta = \min\left(c, \frac{p}{\eta_c \eta_d}\right), \qquad c' = c - \delta, \quad p' = p - \eta_c \eta_d \delta. \tag{13}$$

则至少有一个新功率为零，且储能状态增量保持不变。对原有购电功率保持不变，净供给增加 $(1 - \eta_c\eta_d)\delta$，可由盈余变量吸收；购电费用不变，各功率上界仍满足。因此，至少存在一个不同时充放电的同成本最优解。程序对求解结果实施式(13)，并核验全部时段互斥。

若将来引入弃电上限、售电收益、负价格或额外运行机制，上述等价性需重新判断；必要时可引入二元变量 z_t，设置 $P_{\mathrm{ch},t} \le 5000 z_t$、$P_{\mathrm{dis},t} \le 5000(1 - z_t)$，形成混合整数线性规划。本文主模型不需要二元变量。

输入数据读取后，按变量顺序组成稀疏约束矩阵，调用 `scipy.optimize.linprog` 并设置 `method='highs'`[3, 2]，检查求解成功标志、等式残差和不等式余量，再输出逐时功率、时段电量与分组汇总。固定初始状态后，消去盈余变量和固定为零的紧急变量，核心问题具有 $4 \times 144 = 576$ 个连续变量。

6.1.4 计算结果与调度分析

单日最优购电量为 59482.70 kWh，购电费用为 35126.95 元。指定结果见表3和表4。

表 3 问题 1 指定时段常规购电量（电量单位：kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天常规购电量：59482.70 kWh				全天总费用：35126.95 元	

表 4 问题 1 储能充放电量与边界电量（单位：kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量：6000.00				24:00 储电量：6000.00	

表3按六个指定 10 分钟时段列出常规购电量，末行报告全天总量与费用；表4按六个 4 小时区间列出母线侧充放电量，并报告两端储电量。10:00、14:00 和 20:00 对应时段无需购电，而 12:00–12:10 仍购入 480.41 kWh，说明即使处于日间，也可能因储能补充和跨时段价格差需要购电，不能简单采取“白天一律不购电”的规则。

全天充电 20740.67 kWh、放电 16799.94 kWh，两者之差 3940.73 kWh 为循环损耗。由于日初日末电量相等，$Q_{\mathrm{dis}} = \eta_c \eta_d Q_{\mathrm{ch}}$，该关系与计算结果一致。4:00–8:00 和 8:00–12:00 等分组同时出现正的充电总量和放电总量，是不同 10 分钟时段操作的汇总，并非同一时段同时充放电。

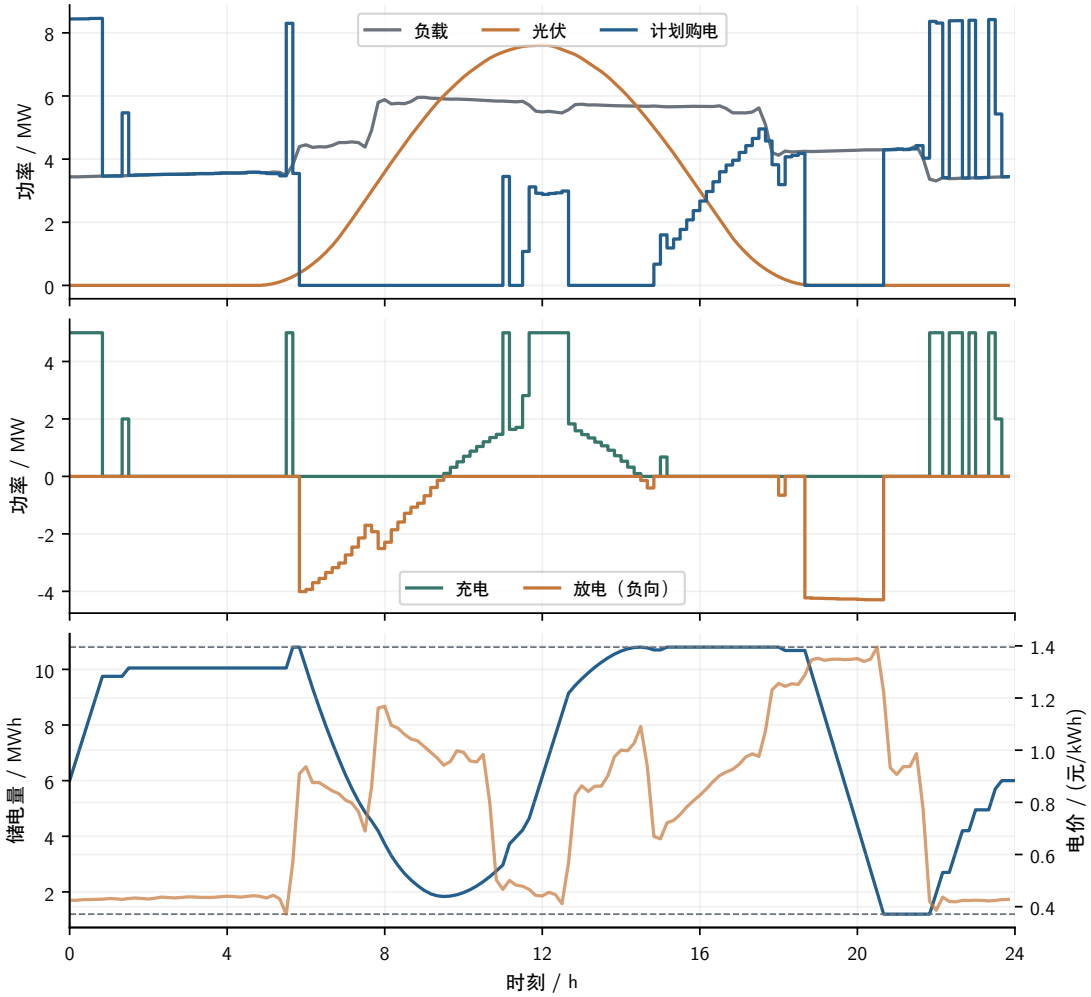


图 2　问题 1 的功率调度、充放电及储能电量轨迹

图2显示，储能在低价阶段及光伏富余阶段充电，在较贵的负荷供给阶段放电。可用电量范围为 9600 kWh，容量边界和 5000 kW 功率限制共同限制套利规模。忽略储能、直接购买正净负荷并弃去光伏盈余的基准费用为 48052.05 元，本文模型节省 12925.10 元，降幅为 26.90%。该比较针对相同单日输入和供电要求。

6.2　问题 2：含紧急补电风险的日前调度模型

6.2.1　基于历史误差场景的费用代理模型

若只最小化点预测下的计划费用，预测偏低导致的少量常规电费节省，可能被 5 倍电价紧急补电抵消。因此，日前模型采用共同购电计划、共同名义充放电轨迹，以及各场景的缺额补救变量 $u_{d,t,s}$。目标为

$$\min J_d = \sum_t \lambda_t P^0_{\mathrm{grid},d,t} \Delta t + \frac{1}{S} \sum_s \sum_t 5 \lambda_t u_{d,t,s} \Delta t. \tag{14}$$

对所有 t, s 设置

$$P^{0}_{\mathrm{grid},d,t} + P^{\mathrm{pl}}_{\mathrm{dis},d,t} - P^{\mathrm{pl}}_{\mathrm{ch},d,t} + u_{d,t,s} \geq n^{(0)}_{d,t,s}, \tag{15}$$

$$u_{d,t,s} \geq 0. \tag{16}$$

名义储能仍满足式(8)–(10)，当日初值取实际 $E^{\mathrm{act}}_{d,0}$，不固定日末电量。第一阶段变量是共同的日前安排，第二阶段变量是场景缺额，因此该模型具有两阶段费用代理结构。**场景中的储能计划保持共同，而实际执行采用状态反馈，二者并不完全等价；故 J_d 不能直接用作论文中的实际费用或全局最优值**。实际成本须经全年运行重新核算。

这一模型将“多购少量常规电量”与“承担高价缺额风险”放在同一目标中权衡，兼顾计算效率。零误差场景数设为 1 且保留 5 倍惩罚时，退化为点预测的日前风险基准，供给通常通过较便宜的常规购电满足。

6.2.2 实际反馈、紧急购电与跨日衔接

在执行时段 t，以生效常规功率和实际净负荷计算

$$R_{d,t} = P_{\mathrm{grid},d,t} + P_{\mathrm{pv},d,t} - P_{\mathrm{load},d,t}. \tag{17}$$

若 $R_{d,t} \geq 0$，取

$$P_{\mathrm{ch},d,t} = \min\left\{R_{d,t}, P_{\max}, \frac{E_{\max} - E_{d,t-1}}{\eta_c \Delta t}\right\}, \quad P_{\mathrm{dis},d,t} = P_{\mathrm{em},d,t} = 0, \tag{18}$$

并令 $P_{\mathrm{sp},d,t} = R_{d,t} - P_{\mathrm{ch},d,t}$。若 $R_{d,t} < 0$，则

$$P_{\mathrm{dis},d,t} = \min\left\{-R_{d,t}, P_{\max}, \frac{\eta_d (E_{d,t-1} - E_{\min})}{\Delta t}\right\}, \tag{19}$$

$$P_{\mathrm{em},d,t} = -R_{d,t} - P_{\mathrm{dis},d,t}, \quad P_{\mathrm{ch},d,t} = P_{\mathrm{sp},d,t} = 0. \tag{20}$$

再依据式(8)更新实际状态。该反馈规则不为维持原定充电功率而紧急购电，而是优先满足实际负荷；其计算只依赖当前已观测量。紧急电量和实际费用为

$$Q_{\mathrm{em},d} = \sum_{t} P_{\mathrm{em},d,t} \Delta t, \tag{21}$$

$$C_d = \sum_{t} \lambda_t P^{0}_{\mathrm{grid},d,t} \Delta t + \sum_{t} 5\lambda_t P_{\mathrm{em},d,t} \Delta t. \tag{22}$$

跨日衔接严格采用 $E^{\mathrm{act}}_{d+1,0} = E^{\mathrm{act}}_{d,144}$。1 月 1 日起连续运行，不把 2 月 1 日重新设为 6000 kWh。主方案 2 月 1 日零时电量为 9340.95 kWh，12 月 31 日末为 1552.80 kWh。

6.2.3 指定日期的购电与储能结果

四个指定日期的完整指标见表5。后续各表按题目给定的购电和储能分组形式展示。问题 2 无日内调整，原计划量与最终常规购电量相同；全天总费用含实际紧急补电费。

表 5 问题 2 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	68991.98	68991.98	0.00	41918.85
06-21	36523.04	36523.04	0.00	21502.49
09-23	68842.58	68842.58	1094.87	49439.26
12-21	98355.39	98355.39	0.00	63456.61

表 6　问题 2：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	680.23	14:00–14:10	0.00
16:00–16:10	489.95	18:00–18:10	701.46	20:00–20:10	0.00
全天常规购电量：68991.98 kWh				全天总费用：41918.85 元	

表 7　问题 2：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8169.92	91.76	4:00–8:00	1955.56	5259.45
8:00–12:00	6279.61	2777.38	12:00–16:00	3657.03	362.33
16:00–20:00	260.66	5765.39	20:00–24:00	526.61	3011.58
0:00 储电量：2058.88				24:00 储电量：1636.78	

表 8　问题 2：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	230.93	18:00–18:10	0.00	20:00–20:10	0.00
全天常规购电量：36523.04 kWh				全天总费用：21502.49 元	

表 9　问题 2：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	990.61	57.57	4:00–8:00	1355.74	1719.95
8:00–12:00	8311.86	0.00	12:00–16:00	0.00	0.00
16:00–20:00	148.50	5265.68	20:00–24:00	1278.87	2485.90
0:00 储电量：3182.63				24:00 储电量：3471.76	

表 10　问题 2：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	524.51	14:00–14:10	0.00
16:00–16:10	526.03	18:00–18:10	762.12	20:00–20:10	0.00
全天常规购电量：68842.58 kWh				全天总费用：49439.26 元	

表 11 问题 2：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	9007.23	85.93	4:00–8:00	1885.31	5720.68
8:00–12:00	5978.77	1896.62	12:00–16:00	3538.48	574.24
16:00–20:00	75.44	6048.66	20:00–24:00	232.09	2384.32
0:00 储电量：1246.18				24:00 储电量：1324.59	

表 12 问题 2：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	997.23	14:00–14:10	0.00
16:00–16:10	859.25	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：98355.39 kWh				全天总费用：63456.61 元	

表 13 问题 2：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8514.82	60.47	4:00–8:00	2166.44	1430.21
8:00–12:00	5900.41	7059.00	12:00–16:00	6532.89	2220.11
16:00–20:00	9.18	5612.85	20:00–24:00	538.84	2842.53
0:00 储电量：1963.33				24:00 储电量：1898.36	

表 14 问题 2 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
无	0.00	无	0.00	20:20–21:30	1018.46	无	0.00
–	–	–	–	21:40–22:00	40.39	–	–
–	–	–	–	22:20–22:30	36.02	–	–

6 月 21 日常规购电量为 36523.04 kWh，低于 12 月 21 日的 98355.39 kWh，反映光伏供给与负载季节差异共同改变外购需求。9 月 23 日出现 1094.87 kWh 紧急购电，其中 20:20–21:30 连续事件占 1018.46 kWh，主要发生在光伏退出后的晚间负荷阶段。其实际紧急费用为 6904.60 元，使全天总费用上升至 49439.26 元。其余三个指定日期无紧急事件，并不意味着这些月份完全没有紧急购电。

6.2.4 报告期总体结果与基准比较

2025 年 2 月至 12 月共 334 天、48096 个时段，主方案计划购电 21791003.35 kWh，计划费用 13376186.67 元，紧急购电 85480.79 kWh，紧急费用 487678.99 元，总费用 13863865.66 元。紧急购电涉及 105 天、733 个 10 分钟时段。

表 15　问题 2 是否纳入历史误差风险的比较

方案	计划电量/万 kWh	紧急电量/kWh	紧急天数	总费用/万元
点预测基准	1999.72	674346.09	272	1599.85
问题 2	2179.10	85480.79	105	1386.39

与点预测基准相比，风险代理模型多购常规电量 1793816.15 kWh，但紧急购电减少 588865.30 kWh，实际总费用减少 2134609.17 元。该结果说明，把紧急成本仅作为事后统计会遗漏重要的计划权衡。另一方面，主方案存在 2753492.48 kWh 未利用供给，表明较保守购电及固定名义储能计划也有代价；该量包括弃光和未利用的已付费供给，不能全部解释为光伏弃电量。

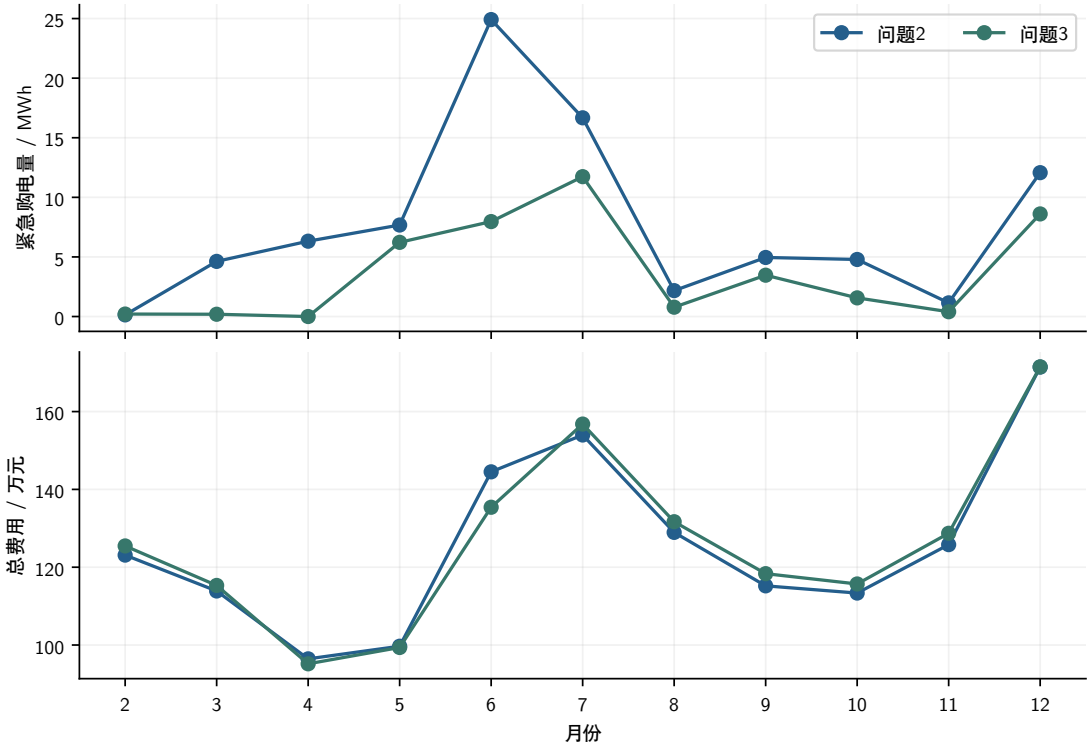


图 3　问题 2 与问题 3 的月度紧急电量和实际总费用

图3中，问题 2 的 6 月紧急购电约 24.91 MWh，高于其他月份，说明夏季光伏较多并不保证预测偏差风险较低。12 月购电总费用约 171.44 万元，体现常规购电需求与电价结构的共同作用，不能只根据紧急电量判断月度经济性。

6.3　问题 3：考虑调整费用的滚动优化模型

6.3.1　计划基准与偏差费用

令 $P^{0}_{\mathrm{grid},d,t}$ 为零时计划，$P^{(k)}_{\mathrm{grid},d,t}$ 为发布时刻 k 得到的调整方案，引入

$$P^{(k)}_{\mathrm{grid},d,t} - P^{0}_{\mathrm{grid},d,t} = y^{+}_{d,t} - y^{-}_{d,t}, \quad y^{+}_{d,t}, y^{-}_{d,t} \geq 0. \tag{23}$$

主口径 A 为原计划全额付费并另加偏差费用：

$$C^{A}_{d} = \sum_{t} \lambda_t \left(P^{0}_{\mathrm{grid},d,t} + 1.5 y^{+}_{d,t} + 0.5 y^{-}_{d,t} + 5 P_{\mathrm{em},d,t} \right) \Delta t. \tag{24}$$

敏感性口径 B 取消下调部分的原电费，再按 50% 支付违约费：

$$C_d^B = \sum_t \lambda_t \left(P^0_{\mathrm{grid},d,t} + 1.5 y^+_{d,t} - 0.5 y^-_{d,t} + 5 P_{\mathrm{em},d,t} \right) \Delta t. \tag{25}$$

例如单价为 1 元/kWh，原计划 100 kWh 下调至 80 kWh 时，口径 A 支付 110 元，口径 B 支付 90 元；等价地，原计划 5 kWh 下调至 4 kWh 时，两口径分别支付 5.5 元和 4.5 元。题目没有明确取消部分是否退回原计划电费，因此不能由原文唯一确定一种解释。本文暂保留加罚口径 A 为主口径，并将退费口径 B 作为并列敏感性结果；二者均重新优化并运行全年反馈，不是只修改显示费用。

在口径 A 且盈余可免费处理的条件下，降低常规购电不仅不能退回原电费，还需支付违约费。保持原购电量并处理盈余始终不劣，因此至少存在 $y^- = 0$ 的最优调整方案。计算中所有主方案的下调量均为零，与该经济逻辑一致。若需要鼓励双向调整，就必须具有退费或其他明确的盈余成本机制。

6.3.2 滚动目标与算法流程

零时使用对应光伏预报与历史负载预测求解日前模型。对 $k = 6, 12, 18$，令剩余时段集合为 $\mathcal{T}_k = \{6k+1, \ldots, 144\}$。原计划费在当前优化中是常数，可从目标中略去。主口径的剩余目标为

$$\min J_{d,k} = \sum_{t \in \mathcal{T}_k} \lambda_t (1.5 y^+_{d,t} + 0.5 y^-_{d,t}) \Delta t + \frac{1}{S} \sum_s \sum_{t \in \mathcal{T}_k} 5 \lambda_t u_{d,t,s} \Delta t. \tag{26}$$

约束包括式(23)、更新场景下的式(15)、储能动态及功率容量边界，初始状态为发布时刻的实际电量。口径 B 只把目标中的 $+0.5y^-$ 换为 $-0.5y^-$。该分段线性费用可直接通过偏差变量构成 LP。

本策略采用有限日内剩余时域的滚动优化，遵循“更新状态、优化未来、只实施前一段”的控制思想 [4]。具体步骤为：

1. 零时获取当前 SOC、历史负载预测与零时光伏预报，形成原始全天购电计划。
2. 按 10 分钟反馈规则执行到下一预报时刻，记录实际充放电、紧急量与储电量。
3. 读取刚发布的光伏预报，对当日剩余时段插值，重建相同发布时刻的历史误差场景。
4. 保持零时计划基准不变，优化剩余购电量与名义储能安排，仅实施到下一次更新。
5. 日末按各时段最终生效购电量相对零时计划的偏差结算一次费用，并传递实际 SOC。

虽然每次预报覆盖未来 24 小时，本文明确假设滚动优化只使用其中当日剩余部分，而不是把“当天合同边界”当作题目规定。结果文件只记录当天 144 个 10 分钟时段，跨日预报不直接计入当天调整费用；次日零时由新发布的预报和连续传递的实际 SOC 初始化。当前实现未让前一日 18 时的跨日预报额外改变次日计划，这会低估该部分信息的潜在价值。

6.3.3 指定日期与实际执行结果

典型日结果见表16及后续指定格式表。其中“调整后”表示最终生效的常规购电总量，并非相对原计划的差值。

表 16　问题 3 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	71993.11	72006.49	113.04	44809.42
06-21	35054.77	35054.77	0.00	20743.13
09-23	72432.47	72432.47	464.02	47642.07
12-21	99503.29	99503.29	0.00	64085.42

表 17　问题 3：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	674.37	14:00–14:10	0.00
16:00–16:10	680.86	18:00–18:10	701.05	20:00–20:10	0.00
全天常规购电量：72006.49 kWh				全天总费用：44809.42 元	

表 18　问题 3：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8172.73	91.76	4:00–8:00	1738.24	6042.51
8:00–12:00	5861.97	2664.33	12:00–16:00	5119.16	254.72
16:00–20:00	0.00	5626.23	20:00–24:00	526.61	3011.58
0:00 储电量：2056.35				24:00 储电量：1676.37	

表 19　问题 3：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	249.05	18:00–18:10	0.00	20:00–20:10	0.00
全天常规购电量：35054.77 kWh				全天总费用：20743.13 元	

表 20　问题 3：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	790.41	59.57	4:00–8:00	1002.19	1719.95
8:00–12:00	8774.71	0.00	12:00–16:00	0.00	0.00
16:00–20:00	156.93	5265.68	20:00–24:00	1271.09	2485.90
0:00 储电量：3266.66				24:00 储电量：3472.35	

表 21 问题 3：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	468.91	14:00–14:10	0.00
16:00–16:10	609.22	18:00–18:10	773.62	20:00–20:10	0.00
全天常规购电量：72432.47 kWh				全天总费用：47642.07 元	

表 22 问题 3：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8974.28	84.63	4:00–8:00	1909.84	5285.29
8:00–12:00	6320.32	1896.62	12:00–16:00	2880.26	403.62
16:00–20:00	119.53	5834.46	20:00–24:00	243.34	2981.17
0:00 储电量：1246.18				24:00 储电量：1331.44	

表 23 问题 3：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	1066.30	14:00–14:10	0.00
16:00–16:10	873.78	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：99503.29 kWh				全天总费用：64085.42 元	

表 24 问题 3：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8514.23	60.47	4:00–8:00	2166.43	1442.92
8:00–12:00	6319.37	6928.90	12:00–16:00	6008.42	2252.05
16:00–20:00	9.18	5612.85	20:00–24:00	538.84	2842.53
0:00 储电量：1963.87				24:00 储电量：1898.36	

表 25 问题 3 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
9:30–10:00	113.04	无	0.00	20:20–21:10	374.11	无	0.00
–	–	–	–	21:20–21:30	13.08	–	–
–	–	–	–	21:40–22:00	40.81	–	–
–	–	–	–	22:20–22:30	36.02	–	–

表 26　问题 3 2025 年 3 月 20 日指定时段计划与调整对照（kWh）

时段	零时计划	最终生效	差额
7:20–7:30	372.57	379.71	7.14
7:30–7:40	281.79	288.03	6.24
9:00–9:10	0.00	0.00	0.00
12:00–12:10	674.37	674.37	0.00
18:00–18:10	701.05	701.05	0.00
20:00–20:10	0.00	0.00	0.00

3 月 20 日原计划 71993.11 kWh，最终常规购电 72006.49 kWh，上调 13.38 kWh，上调费用 14.64 元；仍在 9:30–10:00 发生 113.04 kWh 紧急补电。由此可见，预报更新可减少风险，却不能完全消除负载和光伏预测误差。

9 月 23 日紧急购电量为 464.02 kWh，比问题 2 的 1094.87 kWh 少 630.85 kWh，总费用也从 49439.26 元降至 47642.07 元。但该日没有生效上调量，改善主要来自零时光伏预报不同导致的日前计划变化，不能把全部差异归因于日内滚动调整。

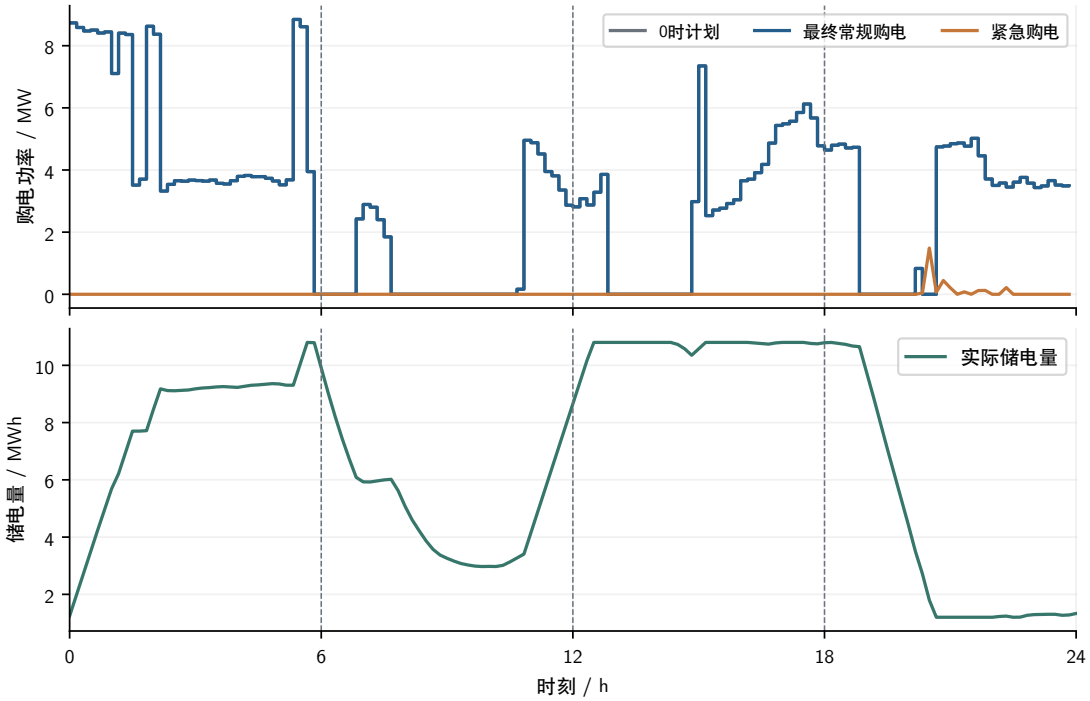


图 4　9 月 23 日问题 3 的购电与储电量；虚线为可更新时刻，该日未触发生效上调

6.3.4　预报更新频率的经济价值

为了单独识别日内更新的作用，使用相同零时预报、相同历史误差窗口和同一结算口径，比较仅零时、0+6、0+12、0+18、0+6+12、0+12+18、0+6+18 及全部四次预报八种组合。各方案均连续运行全年，实际 SOC 允许随其执行结果变化。

表 27 问题 3 不同预报更新频率的比较（加罚主口径）

使用的预报时刻	计划费/元	上调费/元	下调净费/元	紧急量/kWh	紧急费/元	总费用/元	起末 SOC/kWh
仅 0 时	13568930.46	0.00	0.00	77341.11	441424.54	14010355.00	9340.95/1552.80
0、6 时	13564779.71	222258.12	0.00	50079.91	271493.66	14058531.49	9340.95/1552.80
0、12 时	13564736.91	90225.87	0.00	56081.77	296745.51	13951708.29	9340.95/1552.80
0、18 时	13565441.21	67516.12	0.00	50030.50	255551.48	13888508.81	9340.95/1552.80
0、6、12 时	13564732.21	144493.06	0.00	48028.73	257195.37	13966420.64	9340.95/1552.80
0、12、18 时	13564340.98	105527.32	0.00	49190.96	249880.38	13919748.68	9340.95/1552.80
0、6、18 时	13564289.27	241587.20	0.00	41227.19	211053.74	14016930.21	9340.95/1552.80
0、6、12、18 时	13564340.98	159758.44	0.00	41147.37	210396.16	13934495.57	9340.95/1552.80

相较仅使用零时预报，全部四次预报方案节省 75859.43 元，降幅 0.54%，紧急电量从 77341.11 kWh 降至 41147.37 kWh，降幅 46.80%。消融结果显示，0+18 时组合总费用为 13888508.81 元，在本样本中低于全部四次；0+6 和 0+6+18 分别为 14058531.49 元和 14016930.21 元，加入 6 时预报的上调费用较高。0+12+18 为 13919748.68 元，0+6+12 为 13966420.64 元。因而不能仅凭“发布次数越多越好”下结论；在无额外信息成本的前提下，18 时和 12 时预报的边际价值更明显，具体选择仍取决于结算规则与运行目标。

八种组合的计划费、上调费、下调净费、紧急费用、起末储能状态均由同一程序计算，汇总见表27和图5。这里的比较仍是风险代理下的可行策略，不代表全年全局最优。

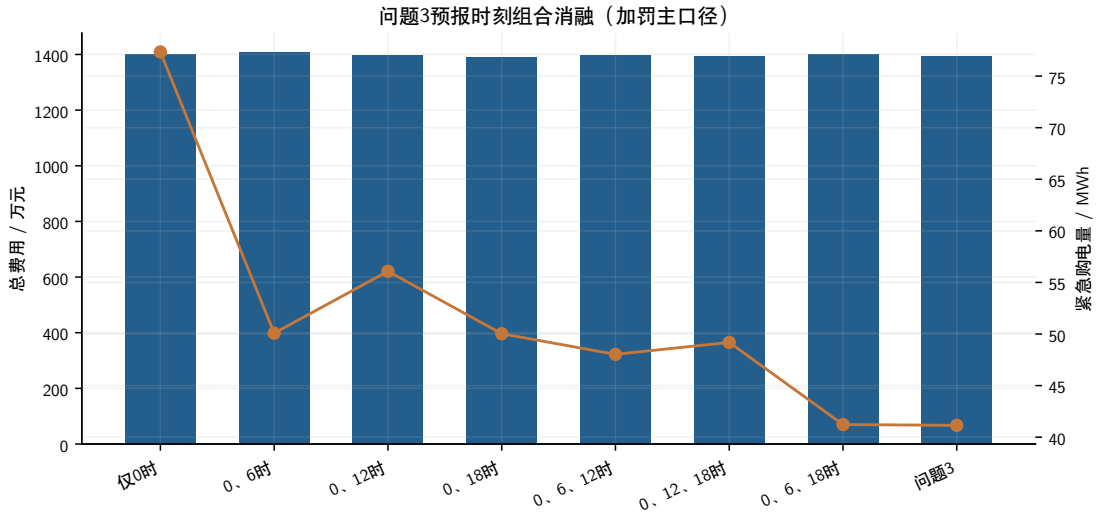


图 5 问题 3 不同预报时刻组合的总费用与紧急购电量（加罚主口径）

问题 3 报告期总费用为 13934495.57 元，比问题 2 多 70629.91 元，约增加 0.51%。虽然紧急费用由 487678.99 元降至 210396.16 元，但较高的原计划费与 159758.44 元上调费抵消了这一收益。该结果说明，本次经验风险代理与给定信息组合不保证滚动方案在所有比较基准上都更便宜，不宜宣称“模型层次越高总费用必然越低”。

为避免只用报告期费用事后选择预报，另按同一目标时段评价附件 3。2–12 月历史光伏预测全天 MAE 为 153.40 kW，而附件 3 零时预报为 197.49 kW；但在 6–12 时和 12–18 时，分别发布于 6 时、12 时的预报 MAE 降至 184.69 kW 和 140.38 kW（见表28）。这解释了“日前计划成本上升、日内紧急购电下降”的可能机制，但误差指标并非费用指标，不能单独推出经济优越性。

在相同零时预报、误差窗口和结算口径下，四次更新相对仅 0 时方案的费用差按月并不稳定（表29）。334 天中 62 天更便宜、133 天更贵、139 天基本相同，节省主要集

中在 7 月；储能跨日传递使单日比较并非相互独立，因此 0.54% 只能视为本样本、当前策略和结算口径下的估计。

表 28　附件 3 光伏预报误差诊断（2 月 1 日–12 月 31 日）

发布时间	相同目标 6 小时 MAE	RMSE	WAPE	发布后剩余全天 MAE
0 时	10.56	43.67	33.67%	197.49
6 时	184.69	269.63	3.82%	158.52
12 时	140.38	192.78	3.03%	75.25
18 时	10.02	42.92	51.45%	10.02

表 29　问题 3 更新相对仅 0 时的月度差异（元、kWh）

月份	费用差（仅 0 时-四次更新）	紧急电量差（四次-仅 0 时）
2	2068.01	1058.54
3	12763.61	3874.99
4	-2179.30	766.36
5	-5962.77	355.26
6	1841.90	4230.25
7	38524.65	11830.74
8	-1659.95	1834.55
9	10519.29	3655.03
10	6006.51	2216.59
11	-2082.38	1856.31
12	16019.86	4515.11

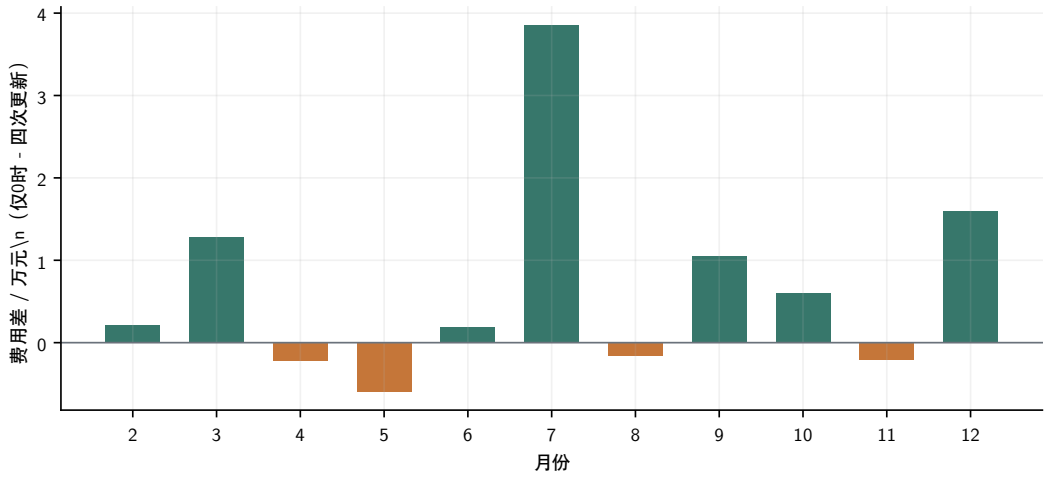


图 6　问题 3 四次更新相对仅 0 时方案的月度费用差；正值表示更新更便宜

6.4 问题 4：波动电价下的调控模型

6.4.1 未来电价未知的可实施模型

价格未知时，以 $\widehat{\lambda}_{d,t}$ 替换式(14)、(26)中的计划价格，得到决策；实际结算时，式(22)、(24)全部使用附件 4 的 $\lambda_{d,t}$。例如问题 4 对应的滚动目标为

$$\min \sum_{t\in\mathcal{T}_k} \widehat{\lambda}_{d,t}(1.5y^{+}_{d,t} + 0.5y^{-}_{d,t})\Delta t + \frac{1}{S}\sum_{s}\sum_{t\in\mathcal{T}_k} 5\widehat{\lambda}_{d,t}u_{d,t,s}\Delta t. \tag{27}$$

物理约束和偏差定义不变。历史价格采用同星期均值，日内光伏更新时继续使用零时形成的价格预测；这是一个实现简单、信息范围明确的价格处理方式，并非完整的日内价格预测器。主方案 `result4-2.xlsx` 与 `result4-3.xlsx` 均采用这一可实施设定。

6.4.2 日前价格已知的对照模型

对照模型在每天零时获知当天附件 4 价格曲线，令 $\widehat{\lambda}_{d,t} = \lambda_{d,t}$，但仍保持未来负载和光伏实际量未知。该方案用于分离价格信息的影响，不应称为全信息完美预测，也不能据其实际费用直接给出全年最优下界。

表 30 波动电价下价格信息的作用

模型与信息	计划费/万元	紧急电量/kWh	总费用/万元	等效循环
日前，价格预测	1410.86	83563.75	1462.14	591.46
日前，价格已知	1398.32	86659.63	1451.68	622.10
滚动，价格预测	1431.32	40537.46	1471.18	588.96
滚动，价格已知	1418.36	41783.49	1458.84	618.73

已知价格条件下，日前模型总费用减少 104569.60 元，滚动模型减少 123349.27 元，分别约占对应价格预测方案费用的 0.72% 和 0.84%。储能等效循环也由 591.46 和 588.96 次增至 622.10 和 618.73 次，表明准确的短时价差信息增加了储能调节活动，但未计入寿命损耗时，更多循环不能直接等同于长期收益更高。

6.4.3 指定日期与费用对比

两套可实施方案的四个指定日期汇总见表31和表33，对应紧急事件见表32和表34。完整的指定时段购电表和 4 小时充放电表列于附录 C，全部 334 天结果写入所要求的两个工作簿。

表 31 问题 4-2 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	69026.66	69026.66	0.00	43411.08
06-21	36837.19	36837.19	0.00	18609.89
09-23	68830.77	68830.77	1076.53	51152.10
12-21	98222.77	98222.77	0.00	74692.77

表 32　问题 4-2 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
无	0.00	无	0.00	20:10–21:30	1000.12	无	0.00
–	–	–	–	21:40–22:00	40.39	–	–
–	–	–	–	22:20–22:30	36.02	–	–

表 33　问题 4-3 典型日购电与费用汇总

日期	原计划/kWh	调整后/kWh	紧急/kWh	总费用/元
03-20	71871.84	71890.12	114.06	46446.36
06-21	35397.64	35397.64	0.00	18134.04
09-23	72586.44	72586.44	466.14	49722.41
12-21	99343.17	99343.17	0.00	75626.96

表 34　问题 4-3 四个指定日期的连续紧急购电事件（kWh）

03-20		06-21		09-23		12-21	
时段	电量	时段	电量	时段	电量	时段	电量
9:30–10:00	114.06	无	0.00	20:10–21:10	376.23	无	0.00
–	–	–	–	21:20–21:30	13.08	–	–
–	–	–	–	21:40–22:00	40.81	–	–
–	–	–	–	22:20–22:30	36.02	–	–

6 月 21 日的波动电价日前策略费用为 18609.89 元，低于重复日内电价时的 21502.49 元；12 月 21 日则从 63456.61 元升至 74692.77 元。电价变化对经济性的影响取决于价格水平、价差与净负荷同时出现的时段，不能仅由“波动增强”判断费用升降。

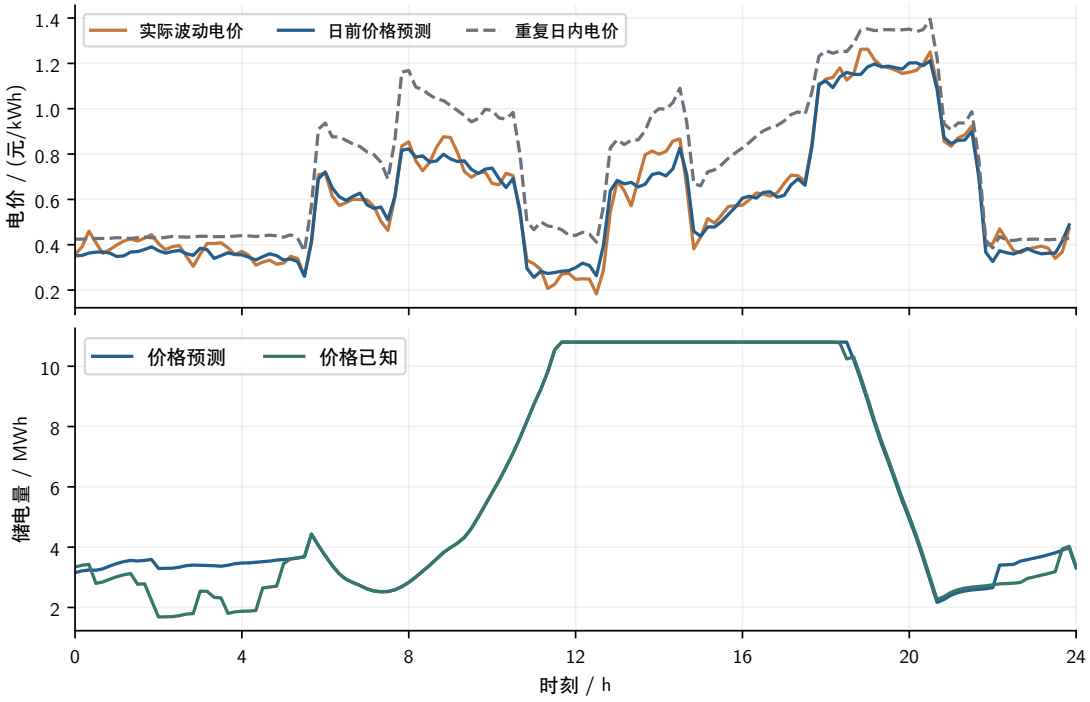


图 7　6 月 21 日电价及问题 4 滚动策略的储能轨迹比较

表 35　2–12 月主模型费用分解（单位：万元）

模型	计划费	上调费	下调净费	紧急费	总费用
问题 2	1337.62	0.00	0.00	48.77	1386.39
问题 3	1356.43	15.98	0.00	21.04	1393.45
问题 4-2	1410.86	0.00	0.00	51.28	1462.14
问题 4-3	1431.32	17.30	0.00	22.56	1471.18

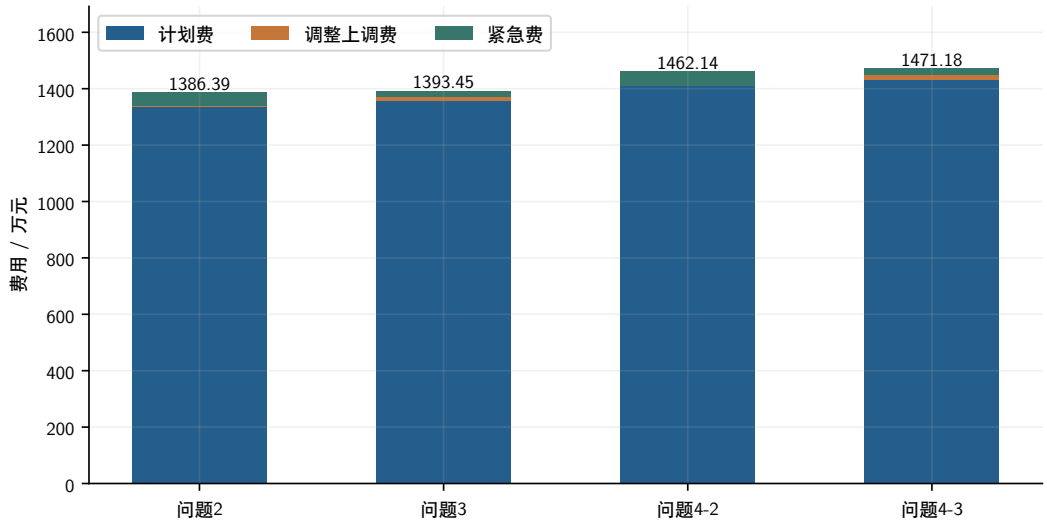


图 8　四套主模型的报告期费用构成

在 2 月至 12 月的实际价格样本下，问题 4 日前与滚动主方案费用分别为 14621415.17 元和 14711757.51 元，均高于重复日内价格对应方案。该差异同时包含实际价格变化与调度变化，不应直接解释为预测方法造成的全部损失。

6.4.4　稀疏模型规模与计算复杂度

剩余时域为 m 个时段、场景数为 S 时，日前模型包含 $(4+S)m$ 个连续变量、m 个状态等式与 Sm 个场景不等式；调整模型再增加 $2m$ 个偏差变量与 m 个计划差等式。矩阵非零元数量和构造空间均为 $O(Sm)$。成熟阶段 $S=15$，全天日前模型有 2736 个连续变量；在 6 时更新时，$m=108$，调整模型有 2268 个变量。

全年逐日前模型调用 365 次 LP，四次预报滚动模型调用 1460 次 LP；仅报告 2 月至 12 月分别对应 334 次和 1336 次，其余用于 1 月份初始化。HiGHS 求解时间取决于稀疏结构、退化性、所选算法和机器条件，不能由上述矩阵规模直接声称整个算法的运行时间也是 $O(Sm)$。实测耗时见表36，不包含依赖安装、绘图、工作簿输出及 LaTeX 编译。

表 36　主模型的实测运行时间（含全年初始化）

模型	LP 次数	LP 耗时/s	全年仿真/s
问题 2	365	4.85	4.94
问题 3	1460	13.95	14.19
问题 4-2	365	5.02	5.11
问题 4-3	1460	13.96	14.20

6.5 模型检验与讨论

6.5.1 物理约束与费用一致性检验

程序逐时独立复算供需平衡、SOC 递推、容量和功率边界，并核查跨日连续性。含新增频率消融在内的 18 组主模型及对照实验的最大实际功率平衡残差不超过 9.1×10^{-13} kW，最大日费用复算差不超过 5.9×10^{-11} 元；全部实际 SOC 位于 1200–10800 kWh，无同一时段同时充放电。用于比较的名义 LP 最大等式与不等式误差亦小于 10^{-7}。

另外，将第 181 天以后的负荷、光伏、价格及预报数据扰动后重新形成预测，前 180 天预测保持完全一致，验证了实现中的时间信息隔离。对 SOC 位于下限且发生缺额、位于上限且存在盈余两种边界单独测试，结果分别触发紧急购电和盈余处理，不突破电量约束。导出文件重新读取后，逐日时段电量和汇总电量、计划费用与对应计算记录一致。

场景虽保存了历史整日误差轨迹，但当前 LP 的补救变量只按逐时缺额计费，共用一条储能计划；因此误差轨迹在不同时间列之间的配对不进入目标。对 2025 年 9 月 23 日逐时独立打乱场景并固定随机种子 20260911 后，LP 代理目标几乎不变，而固定计划的实际反馈费用发生变化（表37）。这项结果是诊断实验，不是新的主模型结果；问题 2–4 的结论应收窄为“逐时缺额风险代理下的可行策略”，不能声称已经利用整日误差相关性或得到全年全局最优。

表 37 场景时间配对诊断：2025 年 9 月 23 日

指标	原始轨迹	逐时打乱	差值
LP 代理目标/元	47479.93	47479.93	-0.00
固定计划实际反馈费/元	47642.07	47840.80	198.74

为检验“缺额时立即放电”是否具有经济性，在固定问题 2 常规购电计划、相同预测和初始状态下，增加一个透明的 36 时段保留规则：每一步为日前预测的未来缺额预留所需电量，再按原有功率和容量边界执行。连续回测结果见表38。该规则在本样本总费用增加 874884.97 元、紧急购电增加 179638.34 kWh，仅 20 天更便宜而 214 天更贵，故它作为负面对照保留，不能据此宣称即时放电已被改进。

表 38 固定常规购电计划下的储能反馈对照（2–12 月）

反馈规则	总费用/元	紧急购电/kWh	末日 SOC/kWh	较便宜日期	较贵日期
即时优先放电	13863865.66	85480.79	1552.80	-	-
36 时段预测缺额保留	14738750.63	265119.13	1552.80	20	214

6.5.2 结算解释、误差窗口与库存差异

表 39 两种结算口径的敏感性比较

价格条件与口径	下调电量/kWh	下调净费/元	总费用/元
重复日内价，主口径	0.00	0.00	13934495.57
重复日内价，退费	1022284.79	-310478.47	13642862.05
波动电价，主口径	0.00	0.00	14711757.51
波动电价，退费	1037194.76	-330099.56	14403672.82

采用退费口径后，问题 3 出现 1022284.79 kWh 下调量，下调净费用为 −310478.47 元，总费用降至 13642862.05 元。相对于主口径下降 291633.53 元，幅度超过仅增加日内更新频率所产生的节省。问题 4 滚动方案也出现类似变化。因此，实际应用前明确退购结算机制，比单纯增加优化次数更为关键。

表 40 问题 3 历史误差窗口的敏感性比较

历史窗口/天	紧急电量/kWh	总费用/元	2 月初 SOC/kWh	年底 SOC/kWh
7	29979.29	13756752.09	9175.29	1347.37
14	41147.37	13934495.57	9340.95	1552.80
28	48500.07	14006955.24	9353.45	1667.29

7 天窗口的紧急量与费用均低于 14 天主窗口，28 天窗口则较高，说明较早误差对当前分布的代表性可能下降。主窗口预先设为 14 天，7 天结果属于报告期敏感性发现，本文未据报告期最优结果回头重选主模型。若进一步部署，可采用仅基于过去已完成日期的嵌套滚动验证调整窗口。

不同策略在统计期边界保有的储能库存略有差异。为进行辅助比较，可取统一参考价值 $v = 0.9 \times 0.4248$ 元/kWh，定义库存调整费用

$$C_{\mathrm{adj}} = C_{\mathrm{total}} + vE_{32,0} - vE_{365,144}. \tag{28}$$

该量仅作同一库存估值下的比较，不替代题目要求的实际现金费用。两种价格信息方案的边界库存价值差异为百元量级，远小于十万元量级的费用差；风险基准与主模型间亦不因该修正改变节省结论。

七、 模型的评价与改进

7.1 模型评价

7.1.1 模型优点

1. 统一物理约束，按预测、交易和价格机制逐层扩展四问模型。
2. 保持稀疏 LP 形式，支持全年批量求解，计算效率较高。
3. 日前计划计入紧急风险，滚动优化可利用更新预报。
4. 区分预测与实际量，采用实际价格结算，结果可复算。

7.1.2 模型局限

1. 共用储能计划的缺额代理只利用逐时误差边际分布，不等同于利用跨时段轨迹的多阶段最优补救策略。
2. 未计入寿命、温度和可变效率，评价主要反映短期交易成本。
3. 历史误差场景可能随季节失配，不能覆盖全部尾部风险。
4. 即时反馈优先放电，未显式保留后续高价时段库存；简单保留规则在本样本无稳定收益，日内价格预测也保持固定。

7.2 改进方向

7.2.1 不确定性与跨日优化

可加入分场景储能补救及非预见性约束，构建多阶段随机规划或分布鲁棒模型，并通过 24–48 小时跨日时域与终端价值修正短期决策。

7.2.2 寿命成本与预测改进

引入与放电深度和能量吞吐相关的寿命成本；结合可获得的气象和节假日信息改进预测，以历史滚动验证选择模型和窗口。

八、 参考文献

[1] Boyd S, Vandenberghe L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004. https://web.stanford.edu/~boyd/cvxbook/.

[2] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1):119–142. DOI:10.1007/s12532-017-0130-5.

[3] SciPy Developers. scipy.optimize.linprog: Linear programming[EB/OL]. [2026-09-10]. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html.

[4] Rawlings J B, Mayne D Q, Diehl M M. Model Predictive Control: Theory, Computation, and Design, 2nd edition[EB/OL]. Nob Hill Publishing. [2026-09-10]. https://sites.engineering.ucsb.edu/~jbraw/mpc/.

[5] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021. [2026-09-10]. https://otexts.com/fpp3/.

附录 A　支撑材料与程序实现说明

A.1　运行环境与文件组织

本次实测环境为 Python 3.12.14、NumPy 2.5.3、SciPy 1.18.1，使用 openpyxl 读取和填充 Excel 模板，Matplotlib 生成论文图形。程序不依赖 pandas；与采用 pandas 的数据框方案相比，直接读入固定形状数组已足以完成本题计算。Python 3.10 是初始建议环境，本文没有将未实际测试的版本和耗时写成实测结果。依赖文件 `code/requirements.txt` 给出本次复现版本。

文件或目录	内容
`main.tex`, `outline.tex`	完整论文源码与三级提纲源码
`code/model.py`	数据读取、历史预测、场景构造、稀疏 LP、运行反馈与状态核验
`code/run_experiments.py`	单日求解、四套主模型、各类对照实验及计时
`code/export_results.py`	5 个指定工作簿的模板填充及保存后复核
`code/make_evidence.py`	由计算结果生成所有数值表格与矢量图形
`code/check_model.py`	功率、SOC、费用、边界及未来数据扰动检查
`code/diagnostics.py`	附件 3 预报误差、场景配对及储能反馈对照诊断
`workbooks/result1.xlsx`	单日计划购电量与储能分组电量
`workbooks/result2.xlsx`	日前计划、实际充放电及连续紧急事件
`workbooks/result3.xlsx`	增加最终生效的调整后常规购电工作表
`workbooks/result4-2.xlsx`, `result4-3.xlsx`	价格预测主设定下对应问题 2 与 3 的结果
`results/*.json`, `daily_summary.csv`	参数、费用分解、典型日与独立核验记录
`tables/`, `figures/`	论文直接调用的真实结果表及矢量图

A.2　运行流程与关键约束编码

先安装依赖，再依次执行以下命令。其中 `DATA` 表示包含附件 1 至 4 及附件 5 文件夹的原始附件目录；原始附件不修改，也不需要预先填入结果。命令应在论文工程目录运行。

```
python -m pip install -r code/requirements.txt
python code/run_experiments.py --data DATA
python code/export_results.py --data DATA
python code/check_model.py --data DATA
python code/diagnostics.py --data DATA
python code/make_evidence.py
xelatex -interaction=nonstopmode main.tex
xelatex -interaction=nonstopmode main.tex
```

模型按购电、充电、放电、状态和场景缺额的顺序拼接变量。储能动态矩阵的状态部分为单位矩阵减去一阶下对角矩阵，充电与放电系数分别为 $-\eta_c\Delta t$ 和 $\Delta t/\eta_d$。场景功率约束采用重复单位矩阵组合，并将大于等于关系乘以负号转换为 `A_ub @ x <= b_ub`。问题 1 将日末状态上下界均置为 6000，紧急变量上下界均置为 0；其余问题释放日末边界并传递实际初值。

批处理逐日执行，仅在一次优化中构建当日剩余时域的稀疏矩阵，不构造全年大规模稠密矩阵。读取后的数据和运行轨迹保存在数组中，结束后统一导出。各方案的全年仿真耗时见正文表36；具体设备和库版本变化可能导致耗时以及等价最优解的个别时段分配略有变化。

A.3 工作簿口径与结果复现

各结果工作簿均保留题目规定的工作表名称。计划购电量记录零时原始计划，其“全天购电费”仅为原计划费；调整购电量记录最终生效的常规购电量，其费用列为原计划费加偏差净费用，尚未加实际紧急费。紧急购电量按连续 10 分钟事件合并，合并电量取所有组成时段电量之和；没有事件的日期标注“无紧急购电”并填 0。完整的实际费用分解见 `daily_summary.csv`，正文使用总费用口径。

充放电量按实际执行轨迹每 4 小时汇总，0:00 和 24:00 为边界状态，不对这些库存量作时段求和。工作簿保留完整精度的数值，显示四位小数；论文表格显示两位小数，因此个别显示值加总可能存在末位舍入差。表格不是示例数据，均由随附程序的实际运行结果生成。

附录 B　完整可运行程序

B.1　数据、预测、优化与反馈

```python
"""C题：仅使用决策时刻可获得信息的预测、线性规划与执行反馈。"""
from pathlib import Path
from time import perf_counter
import numpy as np
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import eye, diags, hstack, vstack, csr_matrix

DT = 1 / 6
ETA = 0.9
EMIN, EMAX, EINIT, PMAX = 1200., 10800., 6000., 5000.
N = 144
METHODS = ('last', 'mean3', 'mean7', 'ewma14', 'weekday')


def read_rows(path, sheet=0):
    wb = load_workbook(path, read_only=True, data_only=True)
    ws = wb.worksheets[sheet] if isinstance(sheet, int) else wb[sheet]
    rows = list(ws.values)
    wb.close()
    return rows


def load_data(root):
    root = Path(root)
    r = read_rows(root / '附件1.xlsx')
    one = np.asarray([row[1:] for row in r[1:]], float)
    actual = []
    for sheet in ('小区负载', '光伏发电实际功率'):
        rows = read_rows(root / '附件2.xlsx', sheet)
        actual.append(np.asarray([row[1:145] for row in rows[1:]], float))
    dates = [row[0].strftime('%Y-%m-%d') for row in rows[1:]]
    price = np.asarray([row[1:145] for row in read_rows(root/'附件4.xlsx')[1:]], float)
    pv_hour = np.asarray([row[2:26] for row in read_rows(root/'附件3.xlsx')[1:]], float).reshape(365,
     4, 24)
    data = dict(one=one, load=actual[0], pv=actual[1], price=price, pv_hour=pv_hour, dates=dates)
    for key in ('one', 'load', 'pv', 'price', 'pv_hour'):
        assert np.isfinite(data[key]).all(), key + ': missing/nonfinite data'
        assert (data[key] >= 0).all(), key + ': negative data'
    assert one.shape == (144, 3)
    assert all(data[k].shape == (365, 144) for k in ('load', 'pv', 'price'))
    assert len(set(dates)) == 365
    return data


def predict(history, fallback, method):
    k = len(history)
    if k == 0:
        return np.asarray(fallback).copy()
    if method == 'last':
        return history[-1].copy()
    if method == 'weekday':
        idx = [k-lag for lag in (7, 14, 21, 28) if k >= lag]
        if idx:
            return np.mean(history[idx], axis=0)
        return np.mean(history[-min(k, 7):], axis=0)
    if method.startswith('mean'):
        return np.mean(history[-min(k, int(method[4:])):], axis=0)
```

```
    block = history[-min(k, 14):]
    weights = np.exp(-np.arange(len(block)-1, -1, -1)/5)
    return np.average(block, weights=weights, axis=0)


def forecast_all(data):
    """1月15—31日选择方法；1月份实际热启动统一用持久性方法。"""
    forecasts, scores, selected = {}, {}, {}
    for key, fallback in [('load', data['one'][:, 1]), ('pv', data['one'][:, 2]), ('price', data['one'][:, 0])]:
        values = data[key]
        scores[key] = {}
        for method in METHODS:
            errors = np.asarray([predict(values[:d], fallback, method)-values[d] for d in range(14, 31)])
            scores[key][method] = dict(mae=float(np.mean(abs(errors))), rmse=float(np.sqrt(np.mean(errors**2))))
        chosen = min(METHODS, key=lambda m: scores[key][m]['mae'])
        selected[key] = chosen
        forecasts[key] = np.asarray([predict(values[:d], fallback, 'last' if d < 31 else chosen) for d in range(365)])
    # 每次预测只连接已观测到的发布时刻端点和该次发布的未来24个整点预报。
    issued = np.zeros((365, 4, 144))
    for day in range(365):
        for issue in range(4):
            start = issue * 36
            current = data['pv'][day, start-1] if start else (data['pv'][day-1, -1] if day else 0.)
            x = (np.arange(start, 144)+1-start)/6
            issued[day, issue, start:] = np.interp(x, np.arange(25), np.r_[current, data['pv_hour'][day, issue]])
    forecasts['issued'] = issued
    return forecasts, scores, selected


def residual_scenarios(data, forecast, day, start, issue=None, history_days=14):
    """当前净负荷预测 + 最近14天在相同发布时刻产生的整条误差轨迹。"""
    pv_forecast = forecast['pv'] if issue is None else forecast['issued'][:, issue]
    nominal = forecast['load'][day, start:] - pv_forecast[day, start:]
    if history_days == 0 or day == 0:
        return nominal[None, :]
    left = max(0, day-history_days)
    errors = data['load'][left:day, start:]-data['pv'][left:day, start:] \
             -(forecast['load'][left:day, start:]-pv_forecast[left:day, start:])
    return np.vstack((nominal, nominal+errors))


def optimize(scenarios, prices, initial, base=None, settlement='additive', cyclic=False, eta=ETA, pmax=PMAX):
    """稀疏LP。紧急量为场景中的缺额补救变量，实际费用由execute重新核算。"""
    start_time = perf_counter()
    s, n = scenarios.shape
    ident = eye(n, format='csr')
    zero = csr_matrix((n, n))
    delta = ident - diags(np.ones(n-1), -1, shape=(n, n), format='csr')
    extra = 2*n if base is not None else 0
    eq = hstack([zero, -eta*DT*ident, DT/eta*ident, delta, csr_matrix((n, s*n+extra))], format='csr')
    eq_rhs = np.zeros(n)
    eq_rhs[0] = initial
    tile = vstack([ident]*s, format='csr')
    ub = hstack([-tile, tile, -tile, csr_matrix((s*n,n)), -eye(s*n,format='csr'), csr_matrix((s*n, extra))],format='csr')
    c = np.r_[prices*DT if base is None else np.zeros(n), np.zeros(3*n), np.tile(5*prices*DT/s,s)]
```

```
    bounds = [(0,None)]*n + [(0,pmax)]*(2*n) + [(EMIN,EMAX)]*n + [(0,None)]*(s*n)
    if cyclic:
        bounds[4*n-1] = (initial,initial)
    if base is not None:
        adjust_eq = hstack([ident,csr_matrix((n,3*n+s*n)),-ident,ident],format='csr')
        eq = vstack([eq,adjust_eq],format='csr')
        eq_rhs = np.r_[eq_rhs,base]
        c = np.r_[c,1.5*prices*DT,(0.5 if settlement=='additive' else -0.5)*prices*DT]
        bounds += [(0,None)]*(2*n)
    # 单日确定性问题必须由常规购电满足负荷，不使用场景紧急量。
    if cyclic:
        for idx in range(4*n,4*n+s*n):
            bounds[idx]=(0,0)
    result=linprog(c,A_ub=ub,b_ub=-scenarios.ravel(),A_eq=eq,b_eq=eq_rhs,bounds=bounds,method='highs')
    if not result.success:
        raise RuntimeError(result.message)
    g,ch,dis,energy=np.split(result.x[:4*n],4)
    # 精确消除同时充放电：状态增量不变，供给裕量增加，由弃电变量吸收。
    amount=np.minimum(ch,dis/eta**2)
    ch=ch-amount
    dis=dis-eta**2*amount
    assert np.max(np.minimum(ch,dis)) < 1e-6
    return dict(grid=g,ch=ch,dis=dis,energy=energy,objective=float(result.fun),seconds=perf_counter()-start_time,
                residual_eq=float(np.max(np.abs(eq@result.x-eq_rhs))),residual_ub=float(np.max(ub@result.x+scenarios.ravel())))


def execute(grid, load, pv, initial, eta=ETA, pmax=PMAX):
    """电网侧功率跟随控制，先用盈余充电；有缺额时放电，再紧急购电。"""
    n=len(grid)
    ch,dis,emergency,spill=np.zeros((4,n))
    energy=np.zeros(n+1)
    energy[0]=initial
    for t in range(n):
        residual=grid[t]+pv[t]-load[t]
        if residual >= 0:
            ch[t]=min(residual,pmax,max(0,(EMAX-energy[t])/(eta*DT)))
            spill[t]=residual-ch[t]
        else:
            dis[t]=min(-residual,pmax,max(0,(energy[t]-EMIN)*eta/DT))
            emergency[t]=-residual-dis[t]
        energy[t+1]=energy[t]+eta*DT*ch[t]-DT/eta*dis[t]
    return dict(ch=ch,dis=dis,emergency=emergency,spill=spill,energy=energy)


def execute_reserve(grid, load, pv, initial, forecast_load=None, forecast_pv=None,
                    prices=None, eta=ETA, pmax=PMAX, reserve_horizon=36):
    """固定常规购电计划下的可复现储能保留基准。

    反馈仍只使用当前实际负荷/光伏；为下一 ``reserve_horizon`` 个10分钟
    时段预留日前预测的缺额所需电量，避免把电池一次性放空。该规则不改动
    常规购电计划或结算，只改变实际充放电与紧急补电，因而可与 execute
    直接作同初值、同信息范围的连续回测。
    """
    n = len(grid)
    if forecast_load is None or forecast_pv is None:
        forecast_load, forecast_pv = load, pv
    ch, dis, emergency, spill = np.zeros((4, n))
    energy = np.zeros(n + 1)
    energy[0] = initial
```

```
    for t in range(n):
        residual = grid[t] + pv[t] - load[t]
        if residual >= 0:
            ch[t] = min(residual, pmax, max(0, (EMAX-energy[t])/(eta*DT)))
            spill[t] = residual - ch[t]
        else:
            end = min(n, t + reserve_horizon + 1)
            expected_deficit = np.maximum(
                forecast_load[t+1:end] - forecast_pv[t+1:end] - grid[t+1:end], 0.0
            ).sum() * DT / eta
            reserve = min(EMAX-EMIN, expected_deficit)
            available = max(0, (energy[t] - EMIN - reserve) * eta / DT)
            dis[t] = min(-residual, pmax, available)
            emergency[t] = -residual - dis[t]
        energy[t+1] = energy[t] + eta*DT*ch[t] - DT/eta*dis[t]
    return dict(ch=ch, dis=dis, emergency=emergency, spill=spill, energy=energy)


def run_case(data,forecast,kind=2,price_mode='fixed',updates=(0,6,12,18),settlement='additive',
     history_days=14,eta=ETA):
    begin=perf_counter()
    arrays={key:np.zeros((365,144)) for key in ('plan','grid','ch','dis','emergency','spill')}
    arrays['energy']=np.zeros((365,145))
    costs=np.zeros((365,5))  # 计划、上调、下调、紧急、合计
    state=EINIT
    lp_seconds=0.; max_eq=0.; max_ub=0.; solves=0
    issue_numbers=[0] if kind==2 else [h//6 for h in updates]
    for day in range(365):
        realized_price=data['one'][:,0] if price_mode=='fixed' else data['price'][day]
        planning_price=forecast['price'][day] if price_mode=='forecast' else realized_price
        arrays['energy'][day,0]=state
        for i,issue in enumerate(issue_numbers):
            start=issue*36
            end=issue_numbers[i+1]*36 if i+1<len(issue_numbers) else 144
            scenarios=residual_scenarios(data,forecast,day,start,None if kind==2 else issue,
     history_days)
            base=arrays['plan'][day,start:] if start else None
            solution=optimize(scenarios,planning_price[start:],state,base,settlement,eta=eta)
            lp_seconds+=solution['seconds']; solves+=1
            max_eq=max(max_eq,solution['residual_eq']);max_ub=max(max_ub,solution['residual_ub'])
            if start==0:
                arrays['plan'][day]=solution['grid']
            g=solution['grid'][:end-start]
            arrays['grid'][day,start:end]=g
            actual=execute(g,data['load'][day,start:end],data['pv'][day,start:end],state,eta=eta)
            for key in ('ch','dis','emergency','spill'):
                arrays[key][day,start:end]=actual[key]
            arrays['energy'][day,start:end+1]=actual['energy']
            state=actual['energy'][-1]
        diff=arrays['grid'][day]-arrays['plan'][day]
        costs[day,0]=DT*np.dot(realized_price,arrays['plan'][day])
        costs[day,1]=DT*np.dot(1.5*realized_price,np.maximum(diff,0))
        costs[day,2]=DT*np.dot((.5 if settlement=='additive' else -.5)*realized_price,np.maximum(-
     diff,0))
        costs[day,3]=DT*np.dot(5*realized_price,arrays['emergency'][day])
        costs[day,4]=np.sum(costs[day,:4])
    arrays['costs']=costs
    info=dict(kind=kind,price_mode=price_mode,updates=updates,settlement=settlement,history_days=
     history_days,eta=eta,
              seconds=perf_counter()-begin,lp_seconds=lp_seconds,solves=solves,max_eq_residual=max_eq
     ,max_ub_violation=max_ub)
    audit(data,arrays,eta)
```

```
    return arrays,info


def audit(data,arrays,eta=ETA):
    g,ch,dis,em,spill=[arrays[k] for k in ('grid','ch','dis','emergency','spill')]
    e=arrays['energy']
    residual=g+data['pv']+dis+em-data['load']-ch-spill
    assert np.max(abs(residual))<1e-6
    assert np.min(e)>=EMIN-1e-6 and np.max(e)<=EMAX+1e-6
    assert np.max(np.minimum(ch,dis))<1e-6
    assert np.max(ch)<=PMAX+1e-6 and np.max(dis)<=PMAX+1e-6
    assert np.max(abs(np.diff(e,axis=1)-eta*DT*ch+DT/eta*dis))<1e-6
    assert np.max(abs(e[:-1,-1]-e[1:,0]))<1e-6
    assert abs(e[0,0]-EINIT)<1e-6
    assert (em>=-1e-6).all() and (g>=-1e-6).all()


def summarize(arrays,info):
    section=slice(31,365)
    c=arrays['costs'][section].sum(axis=0)
    out=dict(info)
    for name,val in zip(('plan_cost','up_cost','down_cost','emergency_cost','total_cost'),c):
        out[name]=float(val)
    for name in ('plan','grid','ch','dis','emergency','spill'):
        out[name+'_kwh']=float(arrays[name][section].sum()*DT)
    out['emergency_periods']=int(np.sum(arrays['emergency'][section]>1e-6))
    out['emergency_days']=int(np.sum(np.any(arrays['emergency'][section]>1e-6,axis=1)))
    out['start_energy']=float(arrays['energy'][31,0]);out['end_energy']=float(arrays['energy'
    ][-1,-1])
    out['up_kwh']=float(np.maximum(arrays['grid'][section]-arrays['plan'][section],0).sum()*DT)
    out['down_kwh']=float(np.maximum(arrays['plan'][section]-arrays['grid'][section],0).sum()*DT)
    out['equivalent_cycles']=float((info['eta']*out['ch_kwh']+out['dis_kwh']/info['eta'])/(2*(EMAX-
    EMIN)))
    return out
```

B.2 实验驱动程序

```
"""运行全部模型与对照实验。输入原始附件目录，输出可核验的NPZ与JSON。"""
import argparse,json,sys,platform
from pathlib import Path
from time import perf_counter
import numpy as np
import scipy
from model import load_data,forecast_all,optimize,run_case,summarize,DT,EINIT

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--data',type=Path,required=True)
    parser.add_argument('--out',type=Path,default=Path(__file__).resolve().parents[1]/'results')
    parser.add_argument('--cases',nargs='*')
    args=parser.parse_args();args.out.mkdir(parents=True,exist_ok=True)
    start=perf_counter()
    data=load_data(args.data)
    forecast,scores,selected=forecast_all(data)
    np.savez_compressed(args.out/'inputs_processed.npz',**{k:v for k,v in data.items() if k!='dates'
    },**{'forecast_'+k:v for k,v in forecast.items()})
    metadata={'dates':data['dates'],'selected_forecasts':selected,'january_validation':scores,'python
    ':platform.python_version(),'numpy':np.__version__,'scipy':scipy.__version__}
    metadata['forecast_feb_dec']={}
    for key in ('load','pv','price'):
        diff=forecast[key][31:]-data[key][31:]
```

```
        metadata['forecast_feb_dec'][key]={'mae':float(abs(diff).mean()),'rmse':float(np.sqrt((diff**2).mean())),'wape':float(abs(diff).sum()/data[key][31:].sum())}
    solution=optimize((data['one'][:,1]-data['one'][:,2])[None,:],data['one'][:,0],EINIT,cyclic=True)
    np.savez_compressed(args.out/'q1.npz',**solution,price=data['one'][:,0],load=data['one'][:,1],pv=data['one'][:,2])
    metadata['q1']={k:float(v) for k,v in solution.items() if np.isscalar(v)}
    metadata['q1'].update(grid_kwh=float(solution['grid'].sum()*DT),ch_kwh=float(solution['ch'].sum()*DT),dis_kwh=float(solution['dis'].sum()*DT),end_energy=float(solution['energy'][-1]),net_cost=float(np.maximum(data['one'][:,1]-data['one'][:,2],0)@data['one'][:,0]*DT))
    configs={
      'q2':{},
      'q2_nominal':{'history_days':0},
      'q3':{'kind':3},
      'q3_daily':{'kind':3,'updates':(0,)},
      'q3_noon':{'kind':3,'updates':(0,12)},
      # 预报时刻消融：固定0时基准，逐一识别6、12、18时的独立及组合贡献。
      'q3_06':{'kind':3,'updates':(0,6)},
      'q3_018':{'kind':3,'updates':(0,18)},
      'q3_0612':{'kind':3,'updates':(0,6,12)},
      'q3_01218':{'kind':3,'updates':(0,12,18)},
      'q3_0618':{'kind':3,'updates':(0,6,18)},
      'q3_refund':{'kind':3,'settlement':'refund'},
      'q4_2':{'price_mode':'forecast'},
      'q4_3':{'kind':3,'price_mode':'forecast'},
      'q4_2_known':{'price_mode':'known'},
      'q4_3_known':{'kind':3,'price_mode':'known'},
      'q4_3_refund':{'kind':3,'price_mode':'forecast','settlement':'refund'},
      'q3_history7':{'kind':3,'history_days':7},
      'q3_history28':{'kind':3,'history_days':28},
    }
    summaries={}
    existing=args.out/'summary.json'
    if existing.exists():
        summaries=json.loads(existing.read_text(encoding='utf-8')).get('cases',{})
    for name,config in configs.items():
        if args.cases and name not in args.cases:
            continue
        arrays,info=run_case(data,forecast,**config)
        np.savez_compressed(args.out/(name+'.npz'),**arrays)
        summaries[name]=summarize(arrays,info)
        print(name,json.dumps(summaries[name],ensure_ascii=False),flush=True)
        metadata['cases']=summaries
        metadata['experiment_seconds']=perf_counter()-start
        existing.write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='utf-8')
    print('Completed',selected,flush=True)

if __name__=='__main__':
    if hasattr(sys.stdout,'reconfigure'):sys.stdout.reconfigure(encoding='utf-8')
    main()
```

B.3 结果工作簿导出

```
"""按附件5模板填充5个结果文件；时间标签统一为0:00—24:00。"""
import argparse,json,csv
from pathlib import Path
from copy import copy
from datetime import datetime
import numpy as np
from openpyxl import load_workbook
from openpyxl.styles import Alignment,Font
from model import DT,EINIT
```

```


def clock(t):
    minutes=t*10
    return f'{minutes//60}:{minutes%60:02d}'


def interval(t):
    return clock(t)+'-'+clock(t+1)


def events(power):
    out=[];start=None
    for t in range(145):
        active=t<144 and power[t]>1e-6
        if active and start is None:start=t
        if not active and start is not None:
            out.append((clock(start)+'-'+clock(t),float(power[start:t].sum()*DT)))
            start=None
    return out


def reset_body(ws):
    for merged in list(ws.merged_cells.ranges):ws.unmerge_cells(str(merged))
    styles=[copy(ws.cell(2,c)._style) for c in range(1,ws.max_column+1)]
    if ws.max_row>1:ws.delete_rows(2,ws.max_row-1)
    return styles


def append_row(ws,values,styles):
    ws.append(values)
    r=ws.max_row
    for c,v in enumerate(values,1):
        cell=ws.cell(r,c)
        if c<=len(styles):cell._style=copy(styles[c-1])
        if isinstance(v,(float,np.floating)):cell.number_format='0.0000'
        if isinstance(v,datetime):cell.number_format='yyyy/m/d'
        cell.alignment=Alignment(horizontal='center',vertical='center')
    ws.row_dimensions[r].height=20


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--data',type=Path,required=True)
    parser.add_argument('--root',type=Path,default=Path(__file__).resolve().parents[1]);args=parser.
     parse_args()
    root=args.root;result=root/'results';dest=root/'workbooks';dest.mkdir(exist_ok=True)
    meta=json.loads((result/'summary.json').read_text(encoding='utf-8'));dates=meta['dates']
    q1=np.load(result/'q1.npz')
    wb=load_workbook(args.data/'附件5'/'result1.xlsx')
    ws=wb['计划购电量'];styles=reset_body(ws)
    for t in range(144):append_row(ws,[interval(t),float(q1['grid'][t]*DT)],styles)
    ws=wb['充放电量'];styles=reset_body(ws)
    for block in range(6):
        sl=slice(block*24,(block+1)*24)
        append_row(ws,[f'{block*4}:00-{(block+1)*4}:00',float(q1['ch'][sl].sum()*DT),float(q1['dis'][
     sl].sum()*DT),('0:00' if block==0 else '24:00' if block==1 else None),(EINIT if block==0 else
     float(q1['energy'][-1]) if block==1 else None)],styles)
    wb.save(dest/'result1.xlsx')
    daily=[]
    for case,filename in [('q2','result2.xlsx'),('q3','result3.xlsx'),('q4_2','result4-2.xlsx'),('
     q4_3','result4-3.xlsx')]:
        a=np.load(result/(case+'.npz'));wb=load_workbook(args.data/'附件5'/filename)
```

```
        for sheet,key in [('计划购电量','plan')]+([('调整购电量','grid')] if '调整购电量' in wb.sheetnames else []):
            ws=wb[sheet];styles=reset_body(ws)
            for t in range(144):ws.cell(1,t+2,interval(t))
            for d in range(31,365):
                cost=a['costs'][d,0] if key=='plan' else a['costs'][d,:3].sum()
                append_row(ws,[datetime.fromisoformat(dates[d])]+(a[key][d]*DT).tolist()+[float(a[key][d].sum()*DT),float(cost)],styles)
            ws.freeze_panes='B2'
        ws=wb['充放电量'];styles=reset_body(ws)
        for d in range(31,365):
            for block in range(6):
                sl=slice(block*24,(block+1)*24)
                append_row(ws,[datetime.fromisoformat(dates[d]) if block==0 else None,f'{block*4}:00-{(block+1)*4}:00',float(a['ch'][d,sl].sum()*DT),float(a['dis'][d,sl].sum()*DT),('0:00' if block==0 else '24:00' if block==1 else None),(float(a['energy'][d,0]) if block==0 else float(a['energy'][d,-1]) if block==1 else None)],styles)
        ws.freeze_panes='C2'
        ws=wb['紧急购电量'];styles=reset_body(ws)
        for d in range(31,365):
            for i,(times,kwh) in enumerate(events(a['emergency'][d]) or [('无紧急购电',0.)]):
                append_row(ws,[datetime.fromisoformat(dates[d]) if i==0 else None,times,kwh],styles)
            daily.append([case,dates[d],*a['costs'][d].tolist(),float(a['plan'][d].sum()*DT),float(a['grid'][d].sum()*DT),float(a['emergency'][d].sum()*DT),float(a['energy'][d,0]),float(a['energy'][d,-1])])
        for ws in wb:
            ws.column_dimensions['A'].width=max(ws.column_dimensions['A'].width or 12,16)
            if ws.title=='紧急购电量':ws.column_dimensions['B'].width=24;ws.column_dimensions['C'].width=18
            if ws.title=='充放电量':
                for c in 'BCDEF':ws.column_dimensions[c].width=17
        wb.save(dest/filename)
        # 独立重新读取保存文件，逐日核对电量总和及费用列。
        check=load_workbook(dest/filename,read_only=True,data_only=True)
        matrix=np.asarray([[v for v in row[1:]] for row in list(check['计划购电量'].values)[1:]],float)
        assert np.max(abs(matrix[:,:144].sum(axis=1)-matrix[:,144]))<1e-6
        assert np.max(abs(matrix[:,145]-a['costs'][31:,0]))<1e-6
        check.close()
    with (result/'daily_summary.csv').open('w',newline='',encoding='utf-8-sig') as f:
        writer=csv.writer(f);writer.writerow(['case','date','plan_cost','up_cost','down_cost','emergency_cost','total_cost','plan_kwh','grid_kwh','emergency_kwh','start_energy','end_energy']);writer.writerows(daily)
    print('Exported 5 workbooks; daily energy and cost reconciliation passed.')

if __name__=='__main__':main()
```

B.4 独立核验程序

```
"""对关键物理约束、费用、时间因果性与导出数据做独立检查。"""
import argparse,json
from pathlib import Path
import numpy as np
from model import load_data,forecast_all,run_case,audit,execute,EMIN,EMAX,PMAX,DT,ETA,EINIT

def main():
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,required=True);args=p.parse_args()
    root=Path(__file__).resolve().parents[1]/'results';data=load_data(args.data)
    meta=json.loads((root/'summary.json').read_text(encoding='utf-8'))
    metrics={}
    for name,info in meta['cases'].items():
```

```
        a=dict(np.load(root/(name+'.npz')));audit(data,a,info['eta'])
        prices=np.broadcast_to(data['one'][:,0],(365,144)) if info['price_mode']=='fixed' else data['price']
        diff=a['grid']-a['plan']
        direct=((prices*a['plan'])+1.5*prices*np.maximum(diff,0)+(.5 if info['settlement']=='additive' else -.5)*prices*np.maximum(-diff,0)+5*prices*a['emergency']).sum(axis=1)*DT
        assert np.max(abs(direct-a['costs'][:,4]))<1e-6
        metrics[name]={'max_power_residual':float(abs(a['grid']+data['pv']+a['dis']+a['emergency']-data['load']-a['ch']-a['spill']).max()),'max_cost_residual':float(abs(direct-a['costs'][:,4]).max()),'min_energy':float(a['energy'].min()),'max_energy':float(a['energy'].max()),'simultaneous_periods':int((np.minimum(a['ch'],a['dis'])>1e-6).sum())}
    q=dict(np.load(root/'q1.npz'));energy=np.r_[EINIT,q['energy']]
    assert abs(energy[-1]-EINIT)<1e-6
    assert np.max(abs(np.diff(energy)-ETA*q['ch']*DT+q['dis']*DT/ETA))<1e-6
    assert np.min(q['grid']+q['pv']+q['dis']-q['load']-q['ch'])>=-1e-6
    assert abs(np.dot(q['price'],q['grid'])*DT-float(q['objective']))<1e-6
    # 未来实际数据扰动不应改变此前已发布的预测及方法选择。
    original,_,selected=forecast_all(data)
    changed={k:(v.copy() if isinstance(v,np.ndarray) else v) for k,v in data.items()}
    for key in ('load','pv','price'):changed[key][180:]*=1.5
    changed['pv_hour'][180:]*=.5
    altered,_,selected2=forecast_all(changed)
    assert selected==selected2
    for key in ('load','pv','price','issued'):assert np.array_equal(original[key][:180],altered[key][:180])
    # 进一步重跑日前决策：报告期后数据扰动不能改变此前已形成的计划。
    original_q2 = dict(np.load(root/'q2.npz'))
    altered_q2, _ = run_case(changed, altered, kind=2)
    assert np.array_equal(original_q2['plan'][:180], altered_q2['plan'][:180])
    assert np.array_equal(original_q2['grid'][:180], altered_q2['grid'][:180])
    # 缺电且SOC下限、盈余且SOC上限两种边界。
    low=execute(np.array([0.]),np.array([100.]),np.array([0.]),EMIN)
    high=execute(np.array([100.]),np.array([0.]),np.array([0.]),EMAX)
    assert low['emergency'][0]==100 and low['dis'][0]==0
    assert high['spill'][0]==100 and high['ch'][0]==0
    output={'cases':metrics,'causality_future_perturbation':'passed (forecasts and q2 decisions through day 180)','physical_boundary_checks':'passed','q1_checks':'passed'}
    (root/'verification.json').write_text(json.dumps(output,indent=2),encoding='utf-8')
    print(json.dumps(output,indent=2))
if __name__=='__main__':main()
```

B.5 图表生成程序

```
"""从计算结果自动生成论文数值宏、表格、图形和典型日分析数据。"""
from pathlib import Path
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from model import DT,EINIT,ETA,EMAX,EMIN
from export_results import events

ROOT=Path(__file__).resolve().parents[1]
R=ROOT/'results';T=ROOT/'tables';F=ROOT/'figures'
T.mkdir(exist_ok=True);F.mkdir(exist_ok=True)
S=json.loads((R/'summary.json').read_text(encoding='utf-8'))
D=json.loads((R/'diagnostics.json').read_text(encoding='utf-8'))
CASES={k:dict(np.load(R/(k+'.npz'))) for k in S['cases']}
Q=dict(np.load(R/'q1.npz'));I=dict(np.load(R/'inputs_processed.npz'))
```

```
DATES=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
INDEX=[S['dates'].index(d) for d in DATES]
LABELS={'q2':'问题2','q3':'问题3','q4_2':'问题4-2','q4_3':'问题4-3','q2_nominal':'点预测基准','q3_daily':'仅0时','q3_noon':'0、12时','q3_06':'0、6时','q3_018':'0、18时','q3_0612':'0、6、12时','q3_01218':'0、12、18时','q3_0618':'0、6、18时','q3_refund':'退费口径','q4_2_known':'价格已知：日前','q4_3_known':'价格已知：滚动','q4_3_refund':'波动电价退费口径','q3_history7':'7天窗口','q3_history28':'28天窗口'}
def num(x):return f'{float(x):.2f}'
def save_table(name,caption,headers,rows,label=None):
    text='\\begin{table}[H]\n\\centering\\small\n\\caption{'+caption+'}\n'
    if label:text+='\\label{'+label+'}\n'
    text+='\\begin{tabular}{'+'l'+'r'*(len(headers)-1)+'}\\toprule\n'
    text+=' & '.join(headers)+' \\\\\\midrule\n'
    text+='\n'.join(' & '.join(map(str,row))+' \\\\' for row in rows)
    text+='\n\\bottomrule\\end{tabular}\n\\end{table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')

def purchase(name,caption,g,cost):
    hours=[10,12,14,16,18,20]
    rows=[]
    for line in range(2):
        row=[]
        for h in hours[line*3:line*3+3]:row += [f'{h}:00--{h}:10',num(g[h*6]*DT)]
        rows.append(row)
    text='\\begin{table}[H]\\centering\\small\n\\caption{'+caption+'}\n\\label{tab:'+name+'}\n\\begin{tabular}{lrlrlr}\\toprule\n'
    text+='时段 & 购电量 & 时段 & 购电量 & 时段 & 购电量 \\\\\\midrule\n'
    text+='\n'.join(' & '.join(row)+' \\\\' for row in rows)
    text+='\n\\midrule\n\\multicolumn{3}{l}{全天常规购电量：'+num(g.sum()*DT)+' kWh} & \\multicolumn{3}{r}{全天总费用：'+num(cost)+' 元}\\\\\n\\bottomrule\\end{tabular}\\end{table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')

def storage(name,caption,ch,dis,e0,e1):
    text='\\begin{table}[H]\\centering\\small\n\\caption{'+caption+'}\n\\label{tab:'+name+'}\n\\begin{tabular}{lrrlrr}\\toprule\n时段 & 充电量 & 放电量 & 时段 & 充电量 & 放电量\\\\\\midrule\n'
    for pair in range(3):
        row=[]
        for b in [pair*2,pair*2+1]:
            sl=slice(b*24,(b+1)*24)
            row += [f'{b*4}:00--{(b+1)*4}:00',num(ch[sl].sum()*DT),num(dis[sl].sum()*DT)]
        text+=' & '.join(row)+' \\\\\n'
    text+='\\midrule\\multicolumn{3}{l}{0:00储电量：'+num(e0)+'} & \\multicolumn{3}{r}{24:00储电量：'+num(e1)+'}\\\\\n\\bottomrule\\end{tabular}\\end{table}\n'
    (T/(name+'.tex')).write_text(text,encoding='utf-8')

purchase('q1_purchase','问题1指定时段常规购电量（电量单位：kWh）',Q['grid'],Q['objective'])
storage('q1_storage','问题1储能充放电量与边界电量（单位：kWh）',Q['ch'],Q['dis'],EINIT,Q['energy'][-1])
typical={}
for key in ['q2','q3','q4_2','q4_3']:
    a=CASES[key];typical[key]=[]
    for day,idx in zip(DATES,INDEX):
        token=day[5:].replace('-','')
        purchase(key+'_'+token+'_purchase',LABELS[key]+'：'+day+'指定时段常规购电量（kWh）',a['grid'][idx],a['costs'][idx,4])
        storage(key+'_'+token+'_storage',LABELS[key]+'：'+day+'实际充放电量（kWh）',a['ch'][idx],a['dis'][idx],a['energy'][idx,0],a['energy'][idx,-1])
        typical[key].append(dict(date=day,plan=float(a['plan'][idx].sum()*DT),grid=float(a['grid'][idx].sum()*DT),emergency=float(a['emergency'][idx].sum()*DT),cost=a['costs'][idx].tolist(),start=float(a['energy'][idx,0]),end=float(a['energy'][idx,-1]),events=events(a['emergency'][idx])))
    groups=[events(a['emergency'][idx]) or [('无',0.)] for idx in INDEX]
    text='\\begin{table}[H]\\centering\\scriptsize\n\\caption{'+LABELS[key]+'四个指定日期的连续紧急购电事件（kWh）}\\label{tab:'+key+'_emergency}\n\\begin{tabular}{lrlrlrlr}\\toprule\n'
```

```
    text+=' & '.join('\\multicolumn{2}{c}{'+d[5:]+'}' for d in DATES)+' \\\\\n'
    text+=' & '.join(['时段 & 电量']*4)+' \\\\\\midrule\n'
    for r in range(max(map(len,groups))):
        row=[]
        for g in groups:row += [g[r][0].replace('-','--'),num(g[r][1])] if r<len(g) else ['--','--']
        text+=' & '.join(row)+' \\\\\n'
    text+='\\bottomrule\\end{tabular}\\end{table}\n'
    (T/(key+'_emergency.tex')).write_text(text,encoding='utf-8')
    save_table(key+'_daily_summary',LABELS[key]+'典型日购电与费用汇总',['日期','原计划/kWh','调整后/kWh','紧急/kWh','总费用/元'],[[r['date'][5:],num(r['plan']),num(r['grid']),num(r['emergency']),num(r['cost'][4])] for r in typical[key]],'tab:'+key+'_daily_summary')

# 指定时段同时展示零时计划、最终生效量及差额，避免把调整后数量误作原计划。
idx = INDEX[0]; a = CASES['q3']; ts = [7*6+2, 7*6+3, 9*6, 12*6, 18*6, 20*6]
save_table('q3_plan_adjust', '问题3 2025年3月20日指定时段计划与调整对照（kWh）',
           ['时段','零时计划','最终生效','差额'],
           [[f'{t//6}:{(t%6)*10:02d}--{t//6}:{(t%6+1)*10:02d}', num(a['plan'][idx,t]*DT), num(a['grid'][idx,t]*DT), num((a['grid'][idx,t]-a['plan'][idx,t])*DT)] for t in ts],
           'tab:q3_plan_adjust')

save_table('annual','2--12月主模型费用分解（单位：万元）',['模型','计划费','上调费','下调净费','紧急费','总费用'],[[LABELS[k]]+[num(S['cases'][k][field]/10000) for field in ['plan_cost','up_cost','down_cost','emergency_cost','total_cost']] for k in ['q2','q3','q4_2','q4_3']],'tab:annual')
save_table('risk_compare','问题2是否纳入历史误差风险的比较',['方案','计划电量/万kWh','紧急电量/kWh','紧急天数','总费用/万元'],[[LABELS[k],num(S['cases'][k]['plan_kwh']/1e4),num(S['cases'][k]['emergency_kwh']),S['cases'][k]['emergency_days'],num(S['cases'][k]['total_cost']/1e4)] for k in ['q2_nominal','q2']],'tab:risk')
frequency_keys=['q3_daily','q3_06','q3_noon','q3_018','q3_0612','q3_01218','q3_0618','q3']
save_table('frequency','问题3不同预报更新频率的比较（加罚主口径）',
           ['使用的预报时刻','计划费/元','上调费/元','下调净费/元','紧急量/kWh','紧急费/元','总费用/元','起末SOC/kWh'],
           [[('0、6、12、18时' if k=='q3' else LABELS[k]),num(S['cases'][k]['plan_cost']),num(S['cases'][k]['up_cost']),num(S['cases'][k]['down_cost']),num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['emergency_cost']),num(S['cases'][k]['total_cost']),f"{num(S['cases'][k]['start_energy'])}/{num(S['cases'][k]['end_energy'])}"] for k in frequency_keys],
           'tab:frequency')
save_table('settlement','两种结算口径的敏感性比较',['价格条件与口径','下调电量/kWh','下调净费/元','总费用/元'],[[label,num(S['cases'][k]['down_kwh']),num(S['cases'][k]['down_cost']),num(S['cases'][k]['total_cost'])] for k,label in [('q3','重复日内价，主口径'),('q3_refund','重复日内价，退费'),('q4_3','波动电价，主口径'),('q4_3_refund','波动电价，退费')]],'tab:settlement')
save_table('price_information','波动电价下价格信息的作用',['模型与信息','计划费/万元','紧急电量/kWh','总费用/万元','等效循环'],[[label,num(S['cases'][k]['plan_cost']/1e4),num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['total_cost']/1e4),num(S['cases'][k]['equivalent_cycles'])] for k,label in [('q4_2','日前，价格预测'),('q4_2_known','日前，价格已知'),('q4_3','滚动，价格预测'),('q4_3_known','滚动，价格已知')]],'tab:price_information')
save_table('window','问题3历史误差窗口的敏感性比较',['历史窗口/天','紧急电量/kWh','总费用/元','2月初SOC/kWh','年底SOC/kWh'],[[w,num(S['cases'][k]['emergency_kwh']),num(S['cases'][k]['total_cost']),num(S['cases'][k]['start_energy']),num(S['cases'][k]['end_energy'])] for w,k in [(7,'q3_history7'),(14,'q3'),(28,'q3_history28')]],'tab:window')
save_table('issued_forecast','附件3光伏预报误差诊断（2月1日--12月31日）',['发布时间','相同目标6小时MAE','RMSE','WAPE','发布后剩余全天MAE'],[[f'{k}时',num(v['six_hour']['mae']),num(v['six_hour']['rmse']),f"{100*v['six_hour']['wape']:.2f}\\%",num(v['remaining_day']['mae'])] for k,v in D['issued_pv'].items() if k.isdigit()],'tab:issued_forecast')
save_table('update_robustness','问题3更新相对仅0时的月度差异（元、kWh）',['月份','费用差（仅0时-四次更新）','紧急电量差（四次-仅0时）'],[[m,num(v),num(D['update_emergency_by_month'][m])] for m,v in D['update_cost_by_month'].items()] if 'update_cost_by_month' in D else [],'tab:update_robustness')
fb=D['feedback_benchmark']
save_table('feedback_benchmark','固定常规购电计划下的储能反馈对照（2--12月）',['反馈规则','总费用/元','紧急购电/kWh','末日SOC/kWh','较便宜日期','较贵日期'],[['即时优先放电',num(fb['base_total_cost']),num(fb['base_emergency_kwh']),num(fb['base_end_soc']),'-','-'],['36时段预测缺额保留',num(fb['reserve_total_cost']),num(fb['reserve_emergency_kwh']),num(fb['reserve_end_soc']),fb['cheaper_days'],fb['more_expensive_days']]],'tab:feedback_benchmark')
```

```
save_table('scenario_diagnostic','场景时间配对诊断: 2025年9月23日',['指标','原始轨迹','逐时打乱','差值'],[["LP代理目标/元",num(D['scenario_shuffle']['original_objective']),num(D['scenario_shuffle']['permuted_objective']),num(D['scenario_shuffle']['objective_difference'])],["固定计划实际反馈费/元",num(D['scenario_shuffle']['original_feedback_cost']),num(D['scenario_shuffle']['permuted_feedback_cost']),num(D['scenario_shuffle']['permuted_feedback_cost']-D['scenario_shuffle']['original_feedback_cost'])]],'tab:scenario_diagnostic')
save_table('runtime','主模型的实测运行时间 (含全年初始化) ',['模型','LP次数','LP耗时/s','全年仿真/s'],[[LABELS[k],S['cases'][k]['solves'],num(S['cases'][k]['lp_seconds']),num(S['cases'][k]['seconds'])] for k in ['q2','q3','q4_2','q4_3']],'tab:runtime')
save_table('forecast','历史预测方法的验证与测试误差',['变量','选定方法','1月验证MAE','2--12月MAE','2--12月RMSE','2--12月WAPE'],[[label,method,num(S['january_validation'][k][S['selected_forecasts'][k]]['mae']),num(D['historical'][k]['mae']) if k!='price' else f"{D['historical'][k]['mae']:.4f}",num(D['historical'][k]['rmse']) if k!='price' else f"{D['historical'][k]['rmse']:.4f}",f"{100*D['historical'][k]['wape']:.2f}\\%"] for k,label,method in [('load','负载/kW','同星期均值'),('pv','光伏/kW','7日移动均值'),('price','电价/(元/kWh)','同星期均值')]],'tab:forecast')

font_candidates = [
    Path('/tmp/.TinyTeX/texmf-dist/fonts/opentype/public/fandol/FandolHei-Regular.otf'),
    Path('/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc'),
    Path('/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc'),
]
font_path = next((p for p in font_candidates if p.exists()), None)
if font_path is not None:
    font_manager.fontManager.addfont(str(font_path))
    plot_font = font_manager.FontProperties(fname=str(font_path)).get_name()
else:
    plot_font = 'DejaVu Sans'
plt.rcParams.update({'font.family':plot_font,'font.size':9,'axes.unicode_minus':False,'pdf.fonttype':42,'axes.spines.top':False,'axes.spines.right':False,'axes.grid':True,'grid.alpha':.18,'savefig.bbox':'tight'})
BLUE='#245e8c';ORANGE='#c57639';GREEN='#37776b';GRAY='#6a727b'
hours=np.arange(144)/6
fig,ax=plt.subplots(3,1,figsize=(7.0,6.4),sharex=True,layout='constrained')
ax[0].plot(hours,Q['load']/1000,label='负载',color=GRAY);ax[0].plot(hours,Q['pv']/1000,label='光伏',color=ORANGE);ax[0].step(hours,Q['grid']/1000,label='计划购电',color=BLUE,where='post');ax[0].set_ylabel('功率 / MW');ax[0].legend(ncol=3,loc='upper center',fontsize=8)
ax[1].step(hours,Q['ch']/1000,color=GREEN,label='充电',where='post');ax[1].step(hours,-Q['dis']/1000,color=ORANGE,label='放电 (负向) ',where='post');ax[1].set_ylabel('功率 / MW');ax[1].legend(ncol=2,fontsize=8)
ax[2].plot(np.arange(145)/6,np.r_[EINIT,Q['energy']]/1000,color=BLUE,label='储电量');ax[2].axhline(1.2,color=GRAY,ls='--',lw=.8);ax[2].axhline(10.8,color=GRAY,ls='--',lw=.8);ax[2].set_ylabel('储电量 / MWh');ax[2].set_xlabel('时刻 / h');right=ax[2].twinx();right.plot(hours,Q['price'],color=ORANGE,alpha=.7,label='电价');right.set_ylabel('电价 / (元/kWh)');ax[2].set_xlim(0,24);ax[2].set_xticks(np.arange(0,25,4));fig.savefig(F/'q1_dispatch.pdf');plt.close(fig)

fig,ax=plt.subplots(2,1,figsize=(7,4.8),sharex=True,layout='constrained')
months=np.array([int(d[5:7]) for d in S['dates']]);monthly={}
for k,color in [('q2',BLUE),('q3',GREEN)]:
    em=[CASES[k]['emergency'][months==m].sum()*DT/1000 for m in range(2,13)]
    cost=[CASES[k]['costs'][months==m,4].sum()/10000 for m in range(2,13)]
    ax[0].plot(range(2,13),em,'o-',color=color,label=LABELS[k]);ax[1].plot(range(2,13),cost,'o-',color=color,label=LABELS[k]);monthly[k]={'emergency_mwh':em,'cost_10k':cost}
ax[0].set_ylabel('紧急购电量 / MWh');ax[0].legend(ncol=2);ax[1].set_ylabel('总费用 / 万元');ax[1].set_xlabel('月份');ax[1].set_xticks(range(2,13));fig.savefig(F/'monthly_comparison.pdf');plt.close(fig)
fig, ax = plt.subplots(figsize=(7, 3.2), layout='constrained')
months_report = np.arange(2, 13)
diff_month = np.array([D['update_cost_by_month'][str(m)] for m in months_report]) / 10000
ax.axhline(0, color=GRAY, lw=.8)
ax.bar(months_report, diff_month, color=[GREEN if x >= 0 else ORANGE for x in diff_month], width=.65)
ax.set_xlabel('月份'); ax.set_ylabel('费用差 / 万元\\n (仅0时 - 四次更新) '); ax.set_xticks(months_report)
```

```
fig.savefig(F/'update_robustness.pdf'); plt.close(fig)

# 预报时刻组合消融：费用与紧急购电量同时展示，避免只用三档频率得出结论。
freq_labels=[LABELS[k] for k in frequency_keys]
freq_cost=np.array([S['cases'][k]['total_cost']/10000 for k in frequency_keys])
freq_em=np.array([S['cases'][k]['emergency_kwh']/1000 for k in frequency_keys])
fig, ax1 = plt.subplots(figsize=(8.0,3.8), layout='constrained')
x=np.arange(len(frequency_keys)); ax1.bar(x,freq_cost,color=BLUE,width=.62,label='总费用'); ax1.
    set_ylabel('总费用 / 万元'); ax1.set_xticks(x,freq_labels,rotation=25,ha='right');
ax2=ax1.twinx(); ax2.plot(x,freq_em,'o-',color=ORANGE,label='紧急购电'); ax2.set_ylabel('紧急购电量 /
    MWh')
ax1.set_title('问题3预报时刻组合消融（加罚主口径）'); fig.savefig(F/'q3_frequency.pdf'); plt.close(fig)

idx=INDEX[2];a=CASES['q3'];fig,ax=plt.subplots(2,1,figsize=(7,4.5),sharex=True,layout='constrained')
ax[0].step(hours,a['plan'][idx]/1000,label='0时计划',color=GRAY,where='post');ax[0].step(hours,a['grid
    '][idx]/1000,label='最终常规购电',color=BLUE,where='post');ax[0].plot(hours,a['emergency'][idx
    ]/1000,label='紧急购电',color=ORANGE);ax[0].set_ylabel('购电功率 / MW');ax[0].legend(ncol=3,
    fontsize=8)
ax[1].plot(np.arange(145)/6,a['energy'][idx]/1000,color=GREEN,label='实际储电量');ax[1].set_ylabel('储
    电量 / MWh');ax[1].set_xlabel('时刻 / h');ax[1].legend();ax[1].set_xticks(np.arange(0,25,6));ax
    [1].set_xlim(0,24)
for axis in ax:
    for x in [6,12,18]:axis.axvline(x,color=GRAY,lw=.7,ls='--')
fig.savefig(F/'q3_typical.pdf');plt.close(fig)

idx=INDEX[1];fig,ax=plt.subplots(2,1,figsize=(7,4.5),sharex=True,layout='constrained')
ax[0].plot(hours,I['price'][idx],color=ORANGE,label='实际波动电价');ax[0].plot(hours,I['forecast_price
    '][idx],color=BLUE,label='日前价格预测');ax[0].plot(hours,Q['price'],color=GRAY,ls='--',label='重
    复日内电价');ax[0].set_ylabel('电价 / (元/kWh)');ax[0].legend(ncol=3,fontsize=8)
for k,color in [('q4_3',BLUE),('q4_3_known',GREEN)]:ax[1].plot(np.arange(145)/6,CASES[k]['energy'][
    idx]/1000,color=color,label='价格预测' if k=='q4_3' else '价格已知')
ax[1].set_ylabel('储电量 / MWh');ax[1].set_xlabel('时刻 / h');ax[1].set_xlim(0,24);ax[1].set_xticks(np
    .arange(0,25,4));ax[1].legend(ncol=2);fig.savefig(F/'q4_price.pdf');plt.close(fig)

fig,ax=plt.subplots(figsize=(7,3.5),layout='constrained');keys=['q2','q3','q4_2','q4_3'];x=np.arange
    (4);bottom=np.zeros(4)
for fld,label,color in [('plan_cost','计划费',BLUE),('up_cost','调整上调费',ORANGE),('emergency_cost',
    '紧急费',GREEN)]:
    val=np.array([S['cases'][k][fld]/10000 for k in keys]);ax.bar(x,val,bottom=bottom,label=label,
    color=color,width=.55);bottom+=val
for xx,yy in zip(x,bottom):ax.text(xx,yy+8,f'{yy:.2f}',ha='center',fontsize=9)
ax.set_xticks(x,[LABELS[k] for k in keys]);ax.set_ylabel('费用 / 万元');ax.set_ylim(0,max(bottom)
    *1.15);ax.legend(ncol=3,loc='upper left');fig.savefig(F/'annual_costs.pdf');plt.close(fig)

metrics={'typical':typical,'monthly':monthly,'frequency':{k:{name:S['cases'][k][name] for name in ('
    plan_cost','up_cost','down_cost','emergency_kwh','emergency_cost','total_cost','start_energy','
    end_energy')} for k in frequency_keys},'q1_saving_percent':100*(1-float(Q['objective'])/S['q1']['
    net_cost']),'q2_risk_saving_percent':100*(1-S['cases']['q2']['total_cost']/S['cases']['q2_nominal
    ']['total_cost']),'q3_frequency_saving_percent':100*(1-S['cases']['q3']['total_cost']/S['cases'][
    'q3_daily']['total_cost']),'q3_emergency_reduction_percent':100*(1-S['cases']['q3']['
    emergency_kwh']/S['cases']['q3_daily']['emergency_kwh'])}
(R/'analysis_metrics.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding='utf
    -8')
print(json.dumps({'q1':S['q1'],'metrics':{k:v for k,v in metrics.items() if k not in ['typical','
    monthly']},'typical':typical},ensure_ascii=False))
```

B.6 补充诊断程序

```
"""补充诊断：预报误差、场景配对和储能反馈保留规则。"""
import json
from pathlib import Path
```

```python
import numpy as np

from model import (DT, EINIT, ETA, load_data, forecast_all, residual_scenarios,
                   optimize, execute, execute_reserve)


def metrics(error, truth):
    error = np.asarray(error, float); truth = np.asarray(truth, float)
    return {
        'mae': float(np.mean(np.abs(error))),
        'rmse': float(np.sqrt(np.mean(error**2))),
        'wape': float(np.sum(np.abs(error)) / max(np.sum(np.abs(truth)), 1e-12)),
    }


def run(data_root, out_root):
    out_root = Path(out_root); out_root.mkdir(parents=True, exist_ok=True)
    data = load_data(data_root); fc, _, selected = forecast_all(data)
    report = {'selected_forecasts': selected, 'historical': {}, 'issued_pv': {},
              'scenario_shuffle': {}, 'feedback_benchmark': {}}
    sl = slice(31, 365)
    for key in ('load', 'pv', 'price'):
        report['historical'][key] = metrics(fc[key][sl] - data[key][sl], data[key][sl])
    # 同一目标6小时块和从发布时间起的剩余全天，均按论文插值方法评价。
    for issue in range(4):
        start = issue * 36
        block = slice(start, min(start + 36, 144))
        rest = slice(start, 144)
        truth_block, truth_rest = data['pv'][sl, block], data['pv'][sl, rest]
        pred = fc['issued'][sl, issue]
        report['issued_pv'][str(issue * 6)] = {
            'six_hour': metrics(pred[:, block] - truth_block, truth_block),
            'remaining_day': metrics(pred[:, rest] - truth_rest, truth_rest),
        }
    report['issued_pv']['historical_full_day'] = metrics(
        fc['pv'][sl] - data['pv'][sl], data['pv'][sl])
    report['issued_pv']['zero_full_day'] = metrics(
        fc['issued'][sl, 0] - data['pv'][sl], data['pv'][sl])

    # 指定日期上独立打乱每个时间列，检验目标是否只使用边际分布。
    day = data['dates'].index('2025-09-23')
    scenarios = residual_scenarios(data, fc, day, 0, issue=0)
    rng = np.random.default_rng(20260911)
    shuffled = np.column_stack([rng.permutation(col) for col in scenarios.T])
    q3_for_diag = np.load(out_root / 'q3.npz')
    initial = float(q3_for_diag['energy'][day, 0])
    s1, s2 = optimize(scenarios, data['one'][:, 0], initial), optimize(shuffled, data['one'][:, 0],
     initial)
    f1 = execute(s1['grid'], data['load'][day], data['pv'][day], initial)
    f2 = execute(s2['grid'], data['load'][day], data['pv'][day], initial)
    report['scenario_shuffle'] = {
        'date': data['dates'][day], 'seed': 20260911,
        'original_objective': s1['objective'], 'permuted_objective': s2['objective'],
        'objective_difference': s2['objective'] - s1['objective'],
        'max_grid_difference': float(np.max(np.abs(s2['grid'] - s1['grid']))),
        'original_feedback_cost': float(DT*np.dot(data['one'][:, 0], s1['grid'] + 5*f1['emergency']))
     ,
        'permuted_feedback_cost': float(DT*np.dot(data['one'][:, 0], s2['grid'] + 5*f2['emergency']))
     ,
    }

    # 连续全年回测：固定 q2 常规购电计划，只替换实际储能反馈规则。
```

```
    q2 = np.load(out_root / 'q2.npz')
    q3 = np.load(out_root / 'q3.npz'); q3_daily = np.load(out_root / 'q3_daily.npz')
    delta = q3_daily['costs'][31:, 4] - q3['costs'][31:, 4]
    report['update_cost_by_month'] = {
        str(m): float(delta[[int(x[5:7]) == m for x in data['dates'][31:]]].sum()) for m in range(2, 13)
    }
    report['update_emergency_by_month'] = {
        str(m): float((q3_daily['emergency'][31:][[int(x[5:7]) == m for x in data['dates'][31:]]].sum()
                       - q3['emergency'][31:][[int(x[5:7]) == m for x in data['dates'][31:]]].sum()) * DT)
        for m in range(2, 13)
    }
    report['update_days'] = {
        'cheaper': int(np.sum(delta > 1e-8)),
        'more_expensive': int(np.sum(delta < -1e-8)),
        'same': int(np.sum(np.abs(delta) <= 1e-8)),
    }
    state_g, state_r = EINIT, EINIT
    rows = []
    for d in range(365):
        base = execute(q2['plan'][d], data['load'][d], data['pv'][d], state_g)
        reserve = execute_reserve(q2['plan'][d], data['load'][d], data['pv'][d], state_r,
                                  fc['load'][d], fc['pv'][d])
        p = data['one'][:, 0]
        c_base = DT*np.dot(p, q2['plan'][d] + 5*base['emergency'])
        c_res = DT*np.dot(p, q2['plan'][d] + 5*reserve['emergency'])
        rows.append((d, c_base, c_res, DT*base['emergency'].sum(), DT*reserve['emergency'].sum(),
                     base['energy'][-1], reserve['energy'][-1]))
        state_g, state_r = base['energy'][-1], reserve['energy'][-1]
    rows = np.asarray(rows)
    rr = rows[31:]
    report['feedback_benchmark'] = {
        'rule': '预留未来36个10分钟时段日前预测缺额所需电量',
        'horizon_periods': 36,
        'base_total_cost': float(rr[:, 1].sum()), 'reserve_total_cost': float(rr[:, 2].sum()),
        'cost_difference_reserve_minus_base': float(rr[:, 2].sum()-rr[:, 1].sum()),
        'base_emergency_kwh': float(rr[:, 3].sum()), 'reserve_emergency_kwh': float(rr[:, 4].sum()),
        'emergency_difference_kwh': float(rr[:, 4].sum()-rr[:, 3].sum()),
        'cheaper_days': int(np.sum(rr[:, 2] < rr[:, 1]-1e-8)),
        'more_expensive_days': int(np.sum(rr[:, 2] > rr[:, 1]+1e-8)),
        'same_days': int(np.sum(np.abs(rr[:, 2]-rr[:, 1]) <= 1e-8)),
        'base_end_soc': float(state_g), 'reserve_end_soc': float(state_r),
    }
    # 重新计算两条连续轨迹的全程SOC边界，避免只报告末态。
    state_g, state_r = EINIT, EINIT; mins = [1e9, 1e9]; maxs = [-1e9, -1e9]
    for d in range(365):
        b = execute(q2['plan'][d], data['load'][d], data['pv'][d], state_g)
        r = execute_reserve(q2['plan'][d], data['load'][d], data['pv'][d], state_r, fc['load'][d], fc['pv'][d])
        mins = [min(mins[0], b['energy'].min()), min(mins[1], r['energy'].min())]
        maxs = [max(maxs[0], b['energy'].max()), max(maxs[1], r['energy'].max())]
        state_g, state_r = b['energy'][-1], r['energy'][-1]
    report['feedback_benchmark'].update({'base_min_soc': mins[0], 'reserve_min_soc': mins[1],
                                         'base_max_soc': maxs[0], 'reserve_max_soc': maxs[1]})
    (out_root / 'diagnostics.json').write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding='utf-8')
    print(json.dumps(report, ensure_ascii=False, indent=2))


if __name__ == '__main__':
```

```
    import argparse
    ap = argparse.ArgumentParser(); ap.add_argument('--data', required=True); ap.add_argument('--out', default='results')
    a = ap.parse_args(); run(a.data, a.out)
```

附录 C　问题 4 指定时段的完整结果

C.1　波动电价日前主方案

表 41　问题 4-2：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	680.23	14:00–14:10	0.00
16:00–16:10	489.95	18:00–18:10	700.33	20:00–20:10	0.00
全天常规购电量：69026.66 kWh				全天总费用：43411.08 元	

表 42　问题 4-2：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8877.96	44.72	4:00–8:00	1572.62	5599.32
8:00–12:00	6279.61	2777.38	12:00–16:00	3570.65	366.38
16:00–20:00	223.13	5715.34	20:00–24:00	536.76	3002.67
0:00 储电量：2169.40				24:00 储电量：1673.16	

表 43　问题 4-2：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	230.93	18:00–18:10	484.03	20:00–20:10	0.00
全天常规购电量：36837.19 kWh				全天总费用：18609.89 元	

表 44　问题 4-2：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	762.69	347.46	4:00–8:00	1339.92	1719.95
8:00–12:00	8942.62	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5285.49	20:00–24:00	2025.68	3091.92
0:00 储电量：3156.42				24:00 储电量：3314.88	

表 45　问题 4-2：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	475.98	14:00–14:10	0.00
16:00–16:10	526.03	18:00–18:10	762.12	20:00–20:10	0.00
全天常规购电量：68830.77 kWh				全天总费用：51152.10 元	

表 46 问题 4-2：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8239.20	31.29	4:00–8:00	2571.51	5701.24
8:00–12:00	5978.77	1896.62	12:00–16:00	3475.00	520.38
16:00–20:00	75.44	5901.94	20:00–24:00	275.95	2538.54
0:00 储电量：1246.18				24:00 储电量：1367.11	

表 47 问题 4-2：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	986.10	14:00–14:10	0.00
16:00–16:10	859.25	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：98222.77 kWh				全天总费用：74692.77 元	

表 48 问题 4-2：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	9500.24	60.47	4:00–8:00	1156.20	1430.21
8:00–12:00	5796.89	6955.48	12:00–16:00	5902.39	1729.08
16:00–20:00	7.46	5611.45	20:00–24:00	552.13	2842.53
0:00 储电量：1985.68				24:00 储电量：1910.32	

C.2 波动电价滚动主方案

表 49 问题 4-3：2025-03-20 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	674.37	14:00–14:10	188.29
16:00–16:10	680.86	18:00–18:10	700.12	20:00–20:10	0.00
全天常规购电量：71890.12 kWh				全天总费用：46446.36 元	

表 50 问题 4-3：2025-03-20 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8886.68	44.72	4:00–8:00	852.21	6045.86
8:00–12:00	5861.97	2663.32	12:00–16:00	5119.16	254.72
16:00–20:00	47.76	5585.50	20:00–24:00	536.76	3002.67
0:00 储电量：2161.56				24:00 储电量：1783.66	

表 51　问题 4-3：2025-06-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	249.05	18:00–18:10	479.69	20:00–20:10	0.00
全天常规购电量：35397.64 kWh				全天总费用：18134.04 元	

表 52　问题 4-3：2025-06-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	759.23	330.01	4:00–8:00	1406.10	1719.95
8:00–12:00	8858.35	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5285.49	20:00–24:00	2017.91	3091.92
0:00 储电量：3156.43				24:00 储电量：3307.88	

表 53　问题 4-3：2025-09-23 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	468.91	14:00–14:10	0.00
16:00–16:10	609.22	18:00–18:10	773.62	20:00–20:10	0.00
全天常规购电量：72586.44 kWh				全天总费用：49722.41 元	

表 54　问题 4-3：2025-09-23 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	8249.85	31.29	4:00–8:00	2561.19	5279.44
8:00–12:00	6320.32	1896.62	12:00–16:00	2880.26	403.62
16:00–20:00	119.53	5697.96	20:00–24:00	282.98	3117.67
0:00 储电量：1246.18				24:00 储电量：1367.11	

表 55　问题 4-3：2025-12-21 指定时段常规购电量（kWh）

时段	购电量	时段	购电量	时段	购电量
10:00–10:10	0.00	12:00–12:10	1063.05	14:00–14:10	0.00
16:00–16:10	873.78	18:00–18:10	734.81	20:00–20:10	0.00
全天常规购电量：99343.17 kWh				全天总费用：75626.96 元	

表 56　问题 4-3：2025-12-21 实际充放电量（kWh）

时段	充电量	放电量	时段	充电量	放电量
0:00–4:00	9499.65	60.47	4:00–8:00	1156.19	1442.92
8:00–12:00	6284.77	6894.29	12:00–16:00	5394.09	1761.02
16:00–20:00	7.46	5611.45	20:00–24:00	552.13	2842.53
0:00 储电量：1986.21				24:00 储电量：1910.32	